

CAN VIDEO WORLD MODELS TRACK UNOBSERVED WORLD STATES?

Joonghyuk Shin¹ Yicong Hong² Jaesik Park¹ Xun Huang²

¹Seoul National University, ²Roblox

ABSTRACT

Video world models are increasingly used as simulators, yet visual fidelity alone does not show that a model maintains the hidden state of the world. We examine this gap with an action-conditioned video Shell Game, a visual analog of S_5 state tracking that decouples visual rendering from compositing the hidden state underneath. Bidirectional and autoregressive Transformers, Mamba, and linear attention restricted to nonnegative transition eigenvalues all fit the training horizon of 5 swaps and then fall toward chance on longer swap chains (extrapolation) while still rendering plausible video with additional denoising steps providing no benefit. The pixel-based diffusion target never supervises the unseen hidden state, so the generated frames cannot carry it and the state has to live inside the architecture rather than in the tokens. For a Transformer, that architectural state is only an append-only KV cache, so the model has to re-derive the hidden arrangement from the whole history at every chunk. We find two mechanisms that do extrapolate, and both carry a state across chunks and revise it in place. Linear attention succeeds once its transition eigenvalues may be negative, and TTT with a nonlinear fast weight succeeds by updating the feature map through which it reads its own state. We further examine harder cases in dynamic world exploration tasks, and discuss the broader implications for building stateful video world models.

1 INTRODUCTION

Recent video world models are often judged by whether their generated videos look plausible (Bruce et al., 2024; Parker-Holder et al., 2024). For real world agentic uses, plausibility alone is not enough. A useful world model must remember what is no longer visible, update it as new actions and observations arrive, and recover it when it becomes visually relevant again (Xiao et al., 2025; Lillemark et al., 2026). When the current frame is only a partial observation (e.g., an occluded object, a camera turned away, or a hidden arrangement transformed by actions), the next correct frame is determined not by the present image but by a state carried and updated across the intervening steps.

Recent theoretical work has sharpened when a sequence model can track a long chain of compositional state updates (Merrill et al., 2024). Under standard log-precision assumptions (Merrill & Sabharwal, 2022), fixed-depth Transformers, state-space models, and several linear-attention variants fall inside the parallel complexity class TC^0 ¹, while the word problem over a non-solvable group such as S_5 is NC^1 -hard² and lies outside that class if $TC^0 \neq NC^1$. Such constant-depth parallel backbones cannot, by themselves, compose an arbitrarily long chain of non-commuting state updates. Robust state tracking therefore requires reintroducing serial computation, either through a sufficiently expressive recurrent transition, such as a nonlinear RNN or a linear recurrence with the right algebraic structure (Grazzi et al., 2024), or through chain-of-thought (CoT) scratchpad tokens that externalize one transition per decoding step (Merrill & Sabharwal, 2024). The denoising loop of video diffusion may look like another source of serial compute, but iterative denoising over a constant-depth backbone provably adds no serial depth (Liu et al., 2025). A bidirectional video dif-

¹ TC^0 is the class of problems solved by constant-depth, polynomial-size threshold circuits (a fixed parallel budget independent of input length), contained in NC^1 and conjectured to be strictly smaller.

² NC^1 is the class of problems solved by polynomial-size, $O(\log n)$ -depth Boolean circuits of bounded fan-in; the word problem of such a group is NC^1 -complete (Merrill et al., 2024).

fusion model is thus a TC^0 backbone with a fixed denoising budget: it can fit sequences within its effective parallel depth, but should fail on NC^1 -hard hidden-state tracking once the horizon grows.

In this paper, we present an empirical study of the action-conditioned video Shell Game as a visual analogue of S_5 state tracking. Across our experiments, autoregressive Transformers, SSM (Mamba)-based models, SANA-style Gated DeltaNet variants (Zhu et al., 2026), and bidirectional Transformers all fit the training length but fail to extrapolate, while still rendering plausible frames throughout the action sequence. Autoregressive rollout in DiT does not fix them, since chain-of-thought in language models works by emitting a free-form symbol the model can repurpose as working memory (Merrill & Sabharwal, 2024), whereas a video model’s outputs are pinned by the training loss to the visible scene, so every serial step is spent on rendering, and we observe no scratchpad emerging. What moves between chunks is instead an append-only KV cache that later chunks read but never revise, so each chunk has to re-derive the arrangement from the whole history at a fixed depth, and teacher/diffusion forcing does not make the model’s own generated chunks carry anything else.

Length extrapolation instead comes from carrying a state across chunks and revising it with an expressive transition that can realize one swap, and both matter. Standard linear attentions such as Mamba2 and Gated DeltaNet with nonnegative transition eigenvalues already carry and update such a state at every chunk, but the transitions they admit cannot represent a reflection, whereas the attention backbones carry no such state to begin with. Interestingly, we find two cases that do learn the game, one addressing each of these limitations. For the linear attentions, widening the recurrent transition to admit negative eigenvalues gives them the signed, reflection-like operations a swap requires (Grazzi et al., 2024). For the attention backbones, a test-time-training (TTT) fast weight supplies the state they lack, a small nonlinear MLP (a SwiGLU, $f(x) = W_1 [\text{silu}(W_0 x) \odot (W_2 x)]$) rewritten in place at every chunk. Because its update reaches the feature map (W_0, W_2), not just the readout W_1 , each swap changes how that state is read and written instead of adding one association. This makes the effective kernel history-dependent, placing it beyond plain linear attention. Restricted to W_1 , this update becomes additive under a frozen feature map, in effect static-kernel linear attention (Liu et al., 2026b), and the model again fails past the training length. Our work aims to bridge the state-tracking literature and the recent wave of video world models, and we summarize our contributions as follows:

- We show that current video model backbones supervised solely on visible content cannot carry unobserved states, which makes recent discussions on state tracking directly relevant: how the state is implicitly carried across chunks, and how expressive its transition is.
- On the Shell Game, we find two cases that extrapolate: (1) a recurrent transition widened to admit negative eigenvalues, and (2) a TTT fast weight whose online nonlinear feature-map update makes its effective kernel history-dependent, placing it outside plain linear attention.
- We discuss the implications for more general settings such as static and dynamic world exploration, where state must also be corrected from observations, and what remains open.

2 RELATED WORK

Video World Models. Recent video world models learn to predict future frames from past observations and actions, and now serve tasks as varied as interactive applications (Parker-Holder et al., 2024; Zhang et al., 2025b; He et al., 2025; Shin et al., 2025; Hong et al., 2025; Tang et al., 2025; Wang et al., 2026; Seo et al., 2026), robot planning (Agarwal et al., 2025; Kim et al., 2026; Gao et al., 2026; Ye et al., 2026), and autonomous driving (Ren et al., 2025a; Basant et al., 2026).

Architecturally, these models range from bidirectional Transformers (Peebles & Xie, 2022) to causal Transformers with KV-cache inference (Yin et al., 2024), Mamba (Po et al., 2025; Savov et al., 2025), and more recent DeltaNet-based backbones (Zhu et al., 2026; Zhao et al., 2026). They are typically trained with diffusion or flow-matching losses (Ho et al., 2020; Lipman et al., 2022; Liu et al., 2022b), often combined with few-step distillation (Yin et al., 2023; Song et al., 2023), and conditioned on diverse action signals such as keyboard, camera pose, or mouse drag, frequently alongside text prompts. Self Forcing (Huang et al., 2025b) post-trains a few-step autoregressive diffusion model on its own generated history against a bidirectional distribution-matching teacher, and keeps the long gradient chain tractable by truncating backpropagation through the denoising loop within each chunk, so gradients never run back across the rollout. Building on this common

recipe, much recent work focuses on keeping the generated world consistent, both as stable 3D structure (Ren et al., 2025b; Wu et al., 2025a) and as long-context memory (Wu et al., 2025b).

Yet as these models scale to complex real-world scenarios and simulators, it becomes increasingly hard to handle moving objects under partial observation (Lillemark et al., 2026; Duan et al., 2026; Ma et al., 2026), simulate precise physics (Kang et al., 2024; Aira et al., 2024; Gillman et al., 2025), or maintain a coherent world state across multiple agents (Savva et al., 2026; Liu et al., 2026a).

Linear Attention, Fast Weights, and Test-Time Training. Efficient sequence models replace the growing key-value cache of softmax attention with a fixed-size recurrent state written and read online. Causal linear attention maintains a matrix state $S_t = S_{t-1} + k_t v_t^\top$ read as $o_t = S_t^\top q_t$, an outer-product fast-weight memory (Schlag et al., 2021). Because such additive writes accumulate and collide, the delta rule instead overwrites the stale association and writes only the residual, and DeltaNet makes this update chunk-parallel and trainable at scale (Yang et al., 2024b), while gated variants add data-dependent decay for better memory management (Yang et al., 2024a; Hatamizadeh et al., 2026). Similar recurrence underlies many recent linear-attention backbones (Dao & Gu, 2024; Zhang et al., 2025c), differing mainly in their state transition and read kernel.

Test-time training (TTT) makes the recurrent state the weights of a small inner network updated online during the forward pass, either by a key-value binding loss that associates projected keys with values (TTT-KVB, Sun et al. (2024); Behrouz et al. (2024); Dalal et al. (2025)) or by back-propagating the outer task loss end-to-end through the recurrence onto a subset of the weights (TTT-E2E, Tandon et al. (2025)). LaCT (Zhang et al., 2025a) instantiates this for hardware efficiency by updating the state over large chunks rather than per token, turning the nonlinear fast-weight update into batched, hardware-efficient matrix operations and pairing it with local attention. In autoregressive video diffusion settings, it updates the state on clean frame chunks and reads it to denoise subsequent ones. As we make explicit in Appendix B, a key-value-binding TTT layer with a linear readout reduces to a learned linear-attention update of a recurrent state (Liu et al., 2026b), recovering plain linear attention or the DeltaNet delta rule depending on its inner loss (Sun et al., 2024).

State Tracking and Expressivity. State tracking asks a model to maintain a hidden configuration and update it as a stream of operations arrives, with its canonical hard case, the word problem of a non-solvable group such as S_5 , lying outside the TC^0 reach of fixed-depth Transformers and state-space models (Merrill & Sabharwal, 2022; Merrill et al., 2024). One way to recover this expressivity is to enrich the recurrent update. Writing a linear RNN as $h_t = A(x_t) h_{t-1} + b(x_t)$, its tracking power is set by the spectrum of the transition $A(x_t)$. Widening it to $[-1, 1]$ admits reflection-like transitions with negative eigenvalues, which already suffice for parity (Grazzi et al., 2024), and composing several such reflections per token adds the rotations needed for modular counting and longer permutations (Siems et al., 2025). Nonlinear recurrences recover this power most directly but lose parallel-scan training (Mishra et al., 2026), a tradeoff a growing line of work studies (Siems et al., 2026; Schöne et al., 2025; Merrill et al., 2026). The other approach is to add serial depth at inference, through chain-of-thought tokens (Merrill & Sabharwal, 2024) or by iterating the backbone itself through looped and recursive reasoners (Dehghani et al., 2018; Giannou et al., 2023; Saunshi et al., 2025; Geiping et al., 2025; Wang et al., 2025b; Jolicoeur-Martineau, 2025).

This analysis has largely concerned the language domain, but is now reaching the visual domain. On the generative side, recent analyses place visual autoregressive models within TC^0 (Ke et al., 2025) and show that iterative denoising adds no serial depth of its own (Liu et al., 2025; Chao et al., 2026). On the understanding side, a parallel line benchmarks visual state tracking and finds video models far below humans, from multimodal video question answering (Yu et al., 2026) to shell-game-style entity tracking (Liu & Lee, 2026). We bring a similar lens to action-conditioned video world models, and ask whether they can acquire an unobserved state from a pixel-supervised objective alone.

3 FROM S_5 TO THE VISUAL SHELL GAME

3.1 COMPOSITIONAL STATE TRACKING

State tracking is the word problem of a finite group: the model reads a stream of group elements $g_1, g_2, \dots, g_t$ and must determine their composition $g_1 g_2 \dots g_t$. We instantiate it on the symmetric groups S_3 and S_5 , the permutation groups on three and five elements. S_5 is the smallest sym-

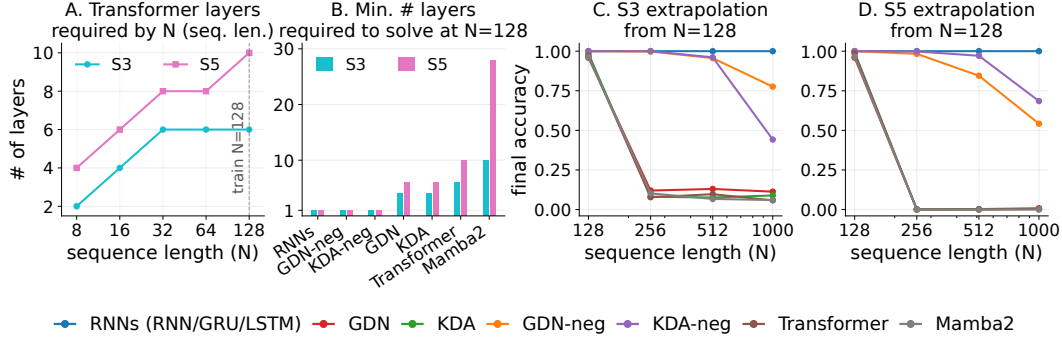


Figure 1: **Pixel-free state tracking on the swap-only S_3 and S_5 word problems.** (A) Minimum number of Transformer layers required to solve a problem of sequence length N . (B) Minimum number of layers required by each architecture at $N=128$. (C) Length extrapolation on S_3 after training at $N=128$. (D) Length extrapolation on S_5 after training at $N=128$.

metric group that is not solvable, containing the simple group A_5 , and this is precisely where the word problem becomes NC^1 -complete (Barrington, 1986) and moves beyond constant-depth parallel models (Liu et al., 2022a), while the solvable S_3 stays easier. We use the *swap* restriction of this problem, following Grazi et al. (2024), where every input is a single transposition and the running state is the permutation formed by composing these swaps. Any permutation of k elements is a product of at most $k - 1$ transpositions (Grazi et al., 2024; Siems et al., 2025), so restricting the inputs to swaps lengthens each chain by only a constant factor and the problem stays NC^1 -complete.

3.2 TOY EXPERIMENTS: STATE TRACKING WITHOUT PIXELS

We first run the synthetic state-tracking probe standard in this literature (Merrill et al., 2024; Grazi et al., 2024), feeding S_3 and S_5 swap chains as raw symbol sequences. For the linear-attention family, whether a single layer can apply one swap is determined by its state-transition matrix. A DeltaNet-style layer carries a matrix state S_t and updates it as

$$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top. \quad (1)$$

The keys k_t are unit-norm, so the transition $A_t = I - \beta_t k_t k_t^\top$ is a generalized Householder matrix with a single non-identity eigenvalue $1 - \beta_t$. The conventional range $\beta_t \in [0, 1]$ confines this eigenvalue to $[0, 1]$, where it can erase and overwrite but never reflect. Widening to $\beta_t \in [0, 2]$ reaches $[-1, 1]$, and at $\beta_t = 2$ the transition becomes the exact reflection that realizes one swap (Grazi et al., 2024). As shown in Fig. 1, nonlinear RNNs apply a swap in a single layer, while linear-attention variants such as Gated DeltaNet (GDN, Yang et al. (2024a)) and Kimi Delta Attention (KDA, Zhang et al. (2025c)) do so only once their eigenvalues are allowed to span $[-1, 1]$, which we mark with a “-neg” suffix throughout. Mamba2, whose transition eigenvalues stay nonnegative, and Transformers, which carry no recurrent transition at all, cannot simulate the composition directly and instead learn a shortcut (Liu et al., 2022a) whose depth grows with the target length and fails to extrapolate past the training length. Consistent with Merrill et al. (2024), even the easier, solvable S_3 already forces the Transformer’s depth to grow with sequence length (Fig. 1). More details are in Appendix C.

3.3 THE VISUAL SHELL GAME: BENCHMARK, MODELS, AND EVALUATION PROTOCOL

Synthetic Data Generation. The Shell Game renders the swap-only S_5 task of Sec. 3.1 as action-conditioned video.³ We adapt the rendering code from Liu & Lee (2026), built on Three.js and WebGL, and add explicit lift and reveal phases together with action annotations (Fig. 2). For k cups, there are $\binom{k}{2}$ pairwise swap actions plus shared lift, lower, and no-op controls. We use $k = 5$ throughout and episodes are rendered at 256×192 resolution. Since the cups are identical and the

³Unlike the full S_5 word problem or its swap-only variant above, which ask for the entire composed permutation, the visual Shell Game only asks for the ball’s position. This is still a single coordinate of the same composed permutation, so producing it depends on the whole swap chain rather than any shortcut.

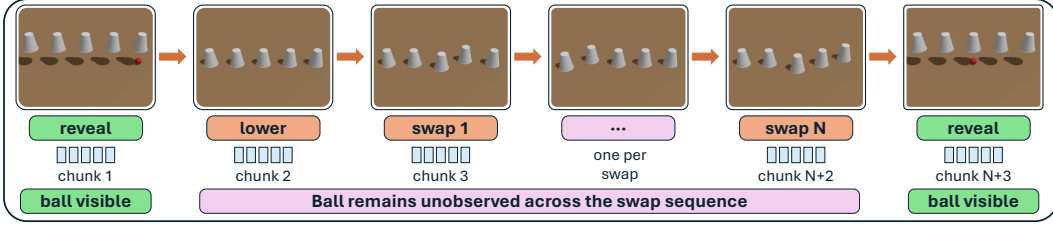


Figure 2: **The visual shell game construction.** An episode first reveals the ball and lowers the cups, then applies a sequence of swaps while the ball remains hidden, and finally reveals its position.

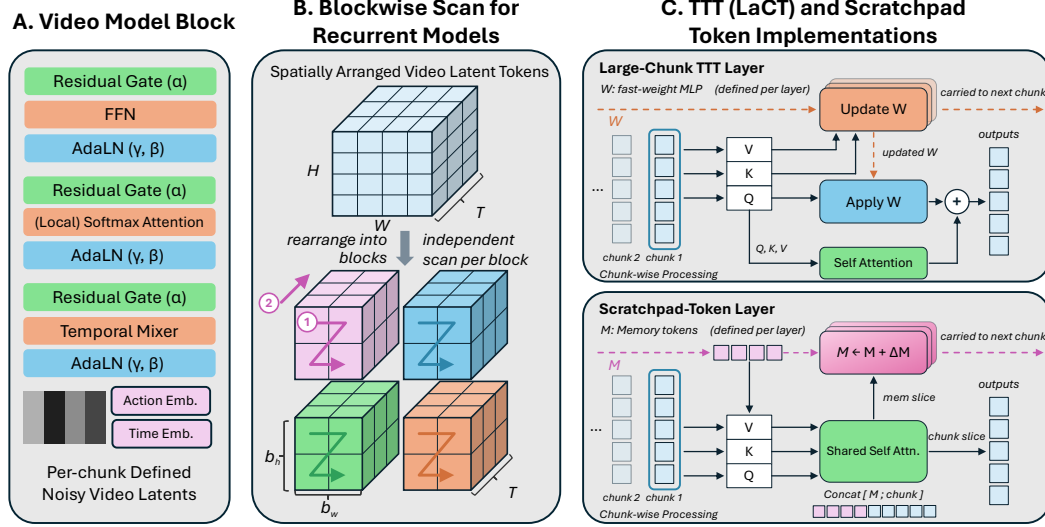


Figure 3: **Architectural details.** (A) A canonical block stacks a temporal mixer, a sliding-window softmax attention with 3D RoPE, and an FFN as residual sublayers (pure-attention baselines drop the temporal mixer). Each sublayer applies standard AdaLN (Peebles & Xie, 2022), with scale γ , shift β , and residual gate α regressed from the action and diffusion-timestep embeddings. (B) Following Po et al. (2025), we first reshape video tokens into an $H \times W \times T$ spatial latent grid and split into $b_h \times b_w \times T$ smaller blocks scanned independently, for better parallelism and a larger effective state. (C) Two examples of persistent per-layer recurrent states carried across autoregressive rollouts. LaCT updates fast-weight MLP W from the chunk’s keys/values, applies it to the queries, and sums with a self-attention branch. The scratchpad keeps M memory tokens updated in place ($M \leftarrow M + \Delta M$) via shared self-attention over a concatenated sequence of $[M; \text{chunk}]$.

camera is fixed, every frame is easy to render and the only difficulty is the hidden arrangement, which is never shown between the two reveals. We align one swap with one autoregressive chunk of five latent frames, so each chunk advances the hidden permutation by one transposition. An episode with N swaps thus spans $N + 3$ chunks in total, including two opening chunks and one final reveal chunk. Further construction and temporal-decoding details are provided in Appendix D.

Models and Training. All variants share a Wan-style latent video diffusion transformer operating on $8 \times 8 \times 4$ -compressed latents from a frozen, pretrained Wan VAE (Wang et al., 2025a). Width is fixed (1152) and depth is set per variant to a comparable $\sim 200\text{M}$ -parameter budget, so the temporal mixer is the main variable. Fig. 3 shows the block layout: a temporal mixer, sliding window attention (SWA) with 3D RoPE, and a feedforward network (FFN) as residual sublayers. The pure-attention DiTs drop the temporal mixer and use softmax attention alone.

We test standard bidirectional and autoregressive Transformers (Chen et al., 2024), non-linear recurrent mixers (RNN, LSTM, GRU), matrix-state linear-attention variants (Mamba2 (Dao & Gu, 2024), GDN (Yang et al., 2024a), KDA (Zhang et al., 2025c)), test-time training with KV-binding (TTT-KVB) represented by LaCT (Zhang et al., 2025a), and the explicit scratchpad (Sec. 5.3). We use a chunk-wise autoregressive setup with a chunk size of 5 latent frames, and restrict SWA to one previous chunk. We write $\text{SWA}\{k\}$ for attention over the current and k previous chunks, giving a total span of $k + 1$ chunks, and $\text{LaCT}\{H\}$ for a LaCT variant with H fast-weight heads.

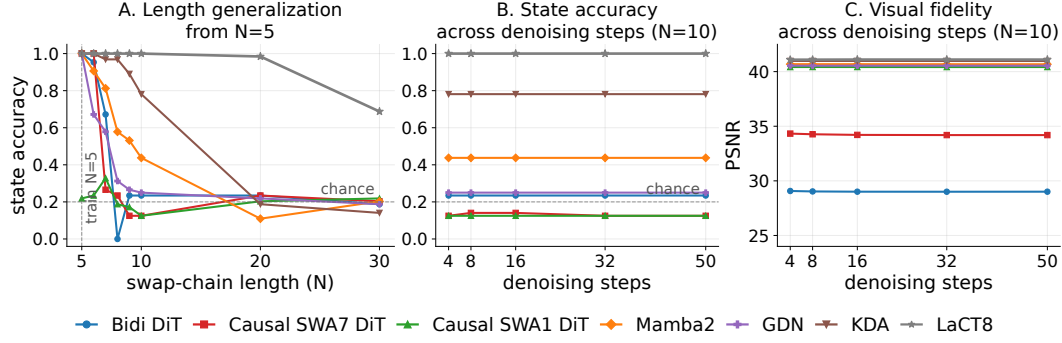


Figure 4: **Shell Game state tracking and visual fidelity under length and denoising-step sweeps.** All recurrent and linear-attention variants share an SWA1 backbone; the causal DiTs are labeled by their window size. **(A)** Length generalization for representative video backbones trained at $N=5$ swaps and evaluated through $N=30$: most standard backbones fit only short horizons and fall toward chance, while LaCT8 retains substantially higher state accuracy. **(B)** At the extrapolation length $N=10$, increasing the denoising budget from 4 to 50 sampling steps does not improve state accuracy. **(C)** The same $N=10$ denoising-step sweep shows high PSNR across models even when state accuracy is near chance, separating visual fidelity from hidden-state tracking.

Following recent work (Po et al., 2025), we apply a spatial block-wise scan to the non-linear recurrent models and linear-attention variants (Fig. 3B). All models are *trained from scratch* on 30,000 sequences of 5 cups and 5 swaps (8 chunks, 157 frames) with diffusion forcing (Chen et al., 2024) under a standard flow-matching schedule. LaCT and the scratchpad variants pack each chunk differently, interleaving its clean copy with the noisy one so that their carried state is updated on clean frames and read while denoising the next chunk, as is conventional for these (Zhang et al., 2025a) (Appendix D.2). We train with a batch size of 16 and a learning rate of 2×10^{-4} for 200K steps in mixed precision, using the NorMuon optimizer (Li et al., 2025), which consistently converged earlier than AdamW under matched settings on both the Shell Game and the original Memory Maze (Fig. A1). Full architecture and training details are given in Appendices D.1 and D.2.

Evaluation Protocol. We train on 30K episodes of $N=5$ swaps and evaluate up to $N=30$, measuring both in-distribution and extrapolation accuracy. With 5 start positions and 10 possible swaps per step, there are 5×10^N distinct length- N episodes, ruling out memorization. We score visual fidelity with PSNR, SSIM, and LPIPS, and state tracking with the final ball-position accuracy (chance $1/5$). We evaluate 64 episodes at chain lengths up to 30 swaps and use 50 sampling steps (Appendix D.3).

4 DO CURRENT VIDEO BACKBONES TRACK HIDDEN STATE?

4.1 STANDARD BACKBONES FAIL: VISUAL FIDELITY DOES NOT IMPLY STATE ACCURACY

Standard backbones learn only shortcuts. Using the setup above, we first test the standard backbones commonly used today: bidirectional and autoregressive (AR) Transformers (Peebles & Xie, 2022; Yin et al., 2024), Mamba2+SWA (Po et al., 2025; Savov et al., 2025), and nonnegative-eigenvalue linear-attention+SWA families (Li et al., 2026; Zhu et al., 2026). Trained for a fixed 200K steps, all full-context attention and recurrent variants except the SWA1 baselines reach 100% accuracy at the in-distribution length, but none of them extrapolates robustly (Fig. 4). As predicted by the toy S_5 analysis (Sec. 3.2), these fixed-depth backbones learn a shortcut for composing a bounded number of swaps and fail once the chain outgrows that depth. Nonetheless, we find that visual fidelity stays high throughout the action sequence, with the model rendering plausible frames even when it fails to track the hidden state. At evaluation we keep each softmax-attention variant within the temporal context it saw during training whenever the architecture allows, so that failures reflect state tracking rather than out-of-distribution RoPE offsets.⁴

⁴The causal full-attention DiT is trained on 8 chunks, so at longer rollouts we cap its attention to the current and 7 previous chunks, matching its training context. With unrestricted context, generation degrades on temporal RoPE offsets never seen in training. SWA variants are evaluated at their training window sizes, and

Increasing denoising steps does not help. As expected from the serial-depth analysis (Liu et al., 2025; Chao et al., 2026), scaling the number of denoising steps does not change accuracy for any of the models above. For the working models, even 4-step sampling matches the accuracy of 50-step sampling. Diffusion timesteps mainly affect visual fidelity, but we suspect our canonical Shell Game videos are simple enough that the model learns easily, so little of that effect shows up here.

4.2 INSPECTING THE AUTOREGRESSIVE CACHE

Autoregressive KV cache is a visual history, not a compact running state. For autoregressive Transformers, KV cache is the main channel that could carry the hidden arrangement across chunks. To probe what it actually carries, we train AR Transformers with different SWA sizes. As shown in Fig. 5, SWA1 through SWA4 fail to track the hidden state even at the training length of 5 swaps, and SWA5 is the first setting in which the final reveal can attend to all five preceding swap chunks. SWA5 and wider windows fit the training length but still fail to extrapolate, including full-context attention. Wider windows cannot fix this because KV entries are fixed once written, so a chunk can only transform what it reads at a higher layer, leaving every chunk to re-derive the arrangement from an append-only history at a fixed depth (Appendix A). A language model would sidestep the problem by writing each intermediate result into its own next token and reading it back one step later, whereas the next chunk here receives a visual latent that carries no target for the hidden ball. All our experiments use a single diffusion forcing pixel-level loss, with chunk-wise causal attention via FlexAttention (Dong et al., 2024), to keep the setup generic. Rollout training (with BPTT), a structured latent state, or task-specific rewards likely help, but we avoid such handcrafting here.

Probing the internal states of autoregressive backbones. We probe the internal states and caches of AR models, and since these are often in complex form (e.g., matrix states or SwiGLU fast weights), we do so indirectly through their readouts. For each temporal chunk t and layer ℓ , we save the model’s own readout y_t^ℓ from the residual update $x_t^{\ell+1} = x_t^\ell + g_t^\ell y_t^\ell$, where g_t^ℓ is the gate.

y_t^ℓ is what the model passes forward to render the current chunk, and is a readout of the model’s internal state given the input query. We also save fixed-bank readouts of the carried state, probing it with a bank of B fixed random Gaussian vectors p_m shared across all chunks. Depending on the backbone, p_m serves as an attention query into the KV cache, $r_{t,m}^{\text{KV}} = \text{Attn}(p_m, K_{\leq t}, V_{\leq t})$, as a linear read of a matrix recurrent state, $r_{t,m}^S = S_t^\top p_m$, or as an input to LaCT’s fast weight, $r_{t,m}^{\text{FW}} = f_{W_t}(p_m)$. We sample readouts from 5 swaps (in-distribution) and 10 swaps (extrapolation) from each model and then train linear probes on either y_t^ℓ or the concatenated bank readout $\{r_{t,m}^\ell\}_{m=1}^B$ to predict the visible swap action and the hidden ball position. Across every backbone and both horizons, a best-layer probe of the current readout y_t^ℓ decodes the visible swap identity at 1.0, whereas hidden-ball accuracy remains far weaker (0.2–0.3 across the models shown, and around 0.2 at $N = 10$). This dissociation is consistent with y_t^ℓ being used primarily to render the current chunk rather than carry a compact persistent state. The fixed-bank readouts instead decode the hidden ball most strongly for the two models that extrapolate, LaCT and GDN-neg (Fig. 5). With $B = 16$, LaCT’s fast-weight state reaches 0.87/0.78 ball accuracy at 5/10 swaps and GDN-neg reaches 0.70/0.46, while the remaining models are substantially weaker. These results suggest that the failing AR back-

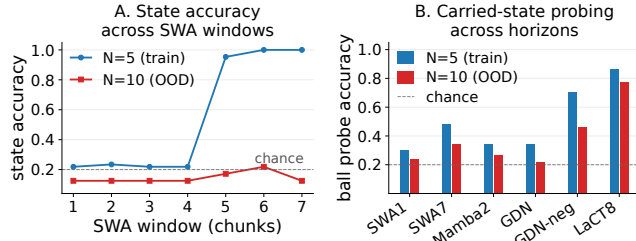


Figure 5: SWA size ablation and hidden-state decodability. The recurrent and linear-attention variants all use an SWA1 backbone. (A) Increasing the SWA window lets a causal DiT fit the training horizon ($N = 5$) once the full swap history is visible, but all windows fail at $N = 10$. SWA7 spans all eight chunks of a training episode and is therefore the widest context available. (B) Best-layer hidden-ball probe accuracy from $B = 16$ fixed-bank reads of the carried state over all swap chunks at $N = 5$ and $N = 10$. Current-readout probes decode the visible swap identity at 1.00 for every shown model and horizon but carry little hidden-ball information (not plotted). GDN-neg and especially LaCT8 retain the strongest decodable state.

a best-layer probe of the current readout y_t^ℓ decodes the visible swap identity at 1.0, whereas hidden-ball accuracy remains far weaker (0.2–0.3 across the models shown, and around 0.2 at $N = 10$). This dissociation is consistent with y_t^ℓ being used primarily to render the current chunk rather than carry a compact persistent state. The fixed-bank readouts instead decode the hidden ball most strongly for the two models that extrapolate, LaCT and GDN-neg (Fig. 5). With $B = 16$, LaCT’s fast-weight state reaches 0.87/0.78 ball accuracy at 5/10 swaps and GDN-neg reaches 0.70/0.46, while the remaining models are substantially weaker. These results suggest that the failing AR back-

we observe no visual-quality degradation. The bidirectional DiT admits no such capping and is evaluated with full context, which often degrades its outputs at longer lengths.

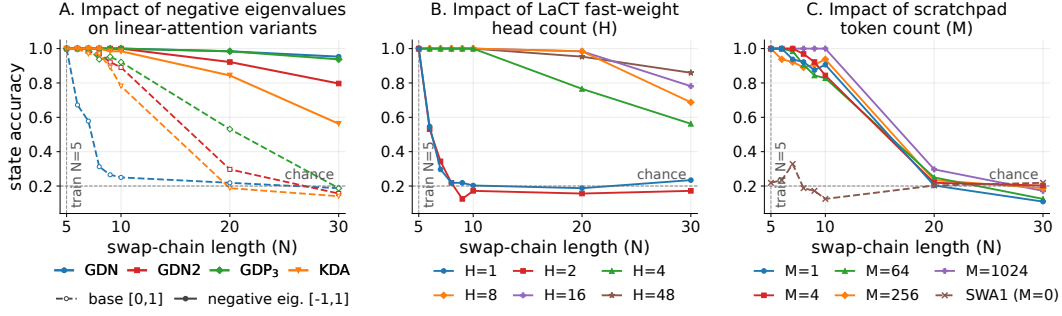


Figure 6: **Length extrapolation of mechanisms that enable state tracking.** All variants share the same SWA1 backbone, with the temporal mechanism added on top. The bare backbone alone (SWA1, $M=0$ in (C)) never rises above chance, failing even at the in-distribution length $N=5$, so it serves as the natural chance-level control. **(A)** Extending the transition eigenvalue range from $[0, 1]$ to $[-1, 1]$ improves state accuracy across linear-attention variants. **(B)** LaCT extrapolation improves with the number of fast-weight heads H . **(C)** Adding explicit per-layer read/writable scratchpad tokens improves extrapolation through $N=10$ but fails by $N=20$.

bones mostly carry action- and retrieval-bound content, while the successful recurrent mechanisms maintain a separate state variable for the hidden swap sequence.

5 MECHANISMS THAT ENABLE STATE TRACKING

From these failures, we hypothesize two key components that enable state tracking: *a state carried across chunks* paired with *an update rule expressive enough to compose the hidden transition on that state in place*. The AR transformers lacked the carried state, and the nonnegative-eigenvalue linear attentions lacked the expressive rule. Here, we show two realizations of this pair, state transitions that allow negative eigenvalues (Sec. 5.1) and TTT-KVB with a nonlinear inner model (Sec. 5.2), together with an explicit-token control that carries a state but only updates it additively (Sec. 5.3).

5.1 NEGATIVE-EIGENVALUE LINEAR ATTENTION

Recalling the DeltaNet-style state update in Eq. 1 from Sec. 3.2, widening $\beta_t \in [0, 1]$ to $\beta_t \in [0, 2]$ allows the transition to reach the full range $[-1, 1]$. We directly enable this across our linear-attention variants, including up-to-date models such as Gated Delta Product (GDP, Siems et al. (2025)) and Gated DeltaNet2 (GDN2, Hatamizadeh et al. (2026)), and find consistent improvements across all variants (Fig. 6), with GDN-neg essentially solving the task (0.95 at $N = 30$).

5.2 NONLINEAR FAST WEIGHTS AS A RECURRENT STATE UPDATE

A second working mechanism comes from LaCT (Zhang et al., 2025a), a TTT-KVB layer whose recurrent state is given by the parameters of a small inner network. We use the bias-free SwiGLU

$$f_W(x) = W_1[\text{silu}(W_0x) \odot (W_2x)], \quad W = (W_0, W_1, W_2). \quad (2)$$

LaCT aggregates the gradient over each clean chunk $\mathcal{C}_t$ and applies one update to the fast weights, which are then read by the following noisy chunk. We use the default negative dot-product KV-binding objective, and ablate it against an MSE objective,

$$\ell_t^{\text{dot}} = - \sum_{i \in \mathcal{C}_t} \langle f_W(k_i), v_i \rangle, \quad \ell_t^{\text{mse}} = \frac{1}{2} \sum_{i \in \mathcal{C}_t} \|f_W(k_i) - v_i\|_2^2. \quad (3)$$

The matrices W_0, W_1, W_2 persist across autoregressive chunks and are updated inside each layer, giving the model an explicit recurrent state that the video objective can learn to use.

More fast-weight heads improve tracking despite smaller raw state size. Fig. 6 shows that one or two heads fail beyond the training length, whereas four or more extrapolate.

At fixed model width D , each head has width $d = D/H$. With SwiGLU expansion ratio r , $W_0, W_2 \in \mathbb{R}^{rd \times d}$ and $W_1 \in \mathbb{R}^{d \times rd}$, so the fast weights carry $H(3rd^2) = 3rD^2/H$ scalars per layer. Increasing H therefore reduces the state size while splitting the update across more independently controlled heads, and the latter appears to matter more for state tracking in this experiment. With a single head, the same fast-weight state must support both rendering and state updates, whereas multiple heads may allow individual channels to specialize in task-relevant state.

Table 1: **LaCT8 fast-weight ablations on the Shell Game.** We report state accuracy at swap-chain length N . All variants use eight fast-weight heads and one online update per chunk. “Freeze” denotes parameters excluded from the online fast-weight update.

Variant	$N=5$	$N=10$	$N=20$	$N=30$
LaCT8 baseline	1.00	1.00	0.98	0.69
+ 1-layer MLP	1.00	0.23	0.23	0.13
+ Freeze W_0, W_2	1.00	0.14	0.30	0.16
+ Freeze W_1	1.00	1.00	0.97	0.64
+ Gradient ascent	1.00	1.00	0.95	0.72
+ MSE loss	0.22	0.16	0.22	0.22

A single-layer linear inner model is insufficient. Replacing LaCT’s SwiGLU with a single-layer MLP preserves recurrent storage and fits the training length, but fails to extrapolate (Tab. 1). For a linear inner map $f_S(x) = S^\top x$, a dot-product gradient step gives $S_t = S_{t-1} + \eta k_t v_t^\top$, the additive outer-product update of plain linear attention (Appendix B). By contrast, the SwiGLU inner model produces a history-dependent feature map that does not admit the same closed-form associative reduction, giving LaCT a richer update class than plain linear attention (Liu et al., 2026b).

State tracking depends on the online feature-map update. Re-writing the SwiGLU inner model,

$$\phi_t(x) = \text{silu}(W_{0,t}x) \odot (W_{2,t}x), \quad f_{W_t}(x) = W_{1,t}\phi_t(x), \quad (4)$$

separates the evolving feature map (W_0, W_2) from the final readout matrix W_1 . As shown in Tab. 1, updating W_1 alone fails, whereas updating W_0 and W_2 with W_1 fixed retains most of it. This shows that state tracking relies on the evolving (W_0, W_2) feature map and the history-dependent kernel it induces, rather than on updates to W_1 alone. By changing the kernel with each swap rather than processing every swap in a fixed feature space, the autoregressive transformers with fast weights can implement the compositional updates required for NC^1 -hard state tracking.

The inner loss does not determine the tracking mechanism. As shown in Tab. 1, reversing the sign of the dot-product update leaves tracking largely unchanged. Consistent with the gradient-ascent observation of Liu et al. (2026b), end-to-end training can absorb the sign of the fast-weight write. We find the MSE inner loss more sensitive. In the token-wise, single-layer linear case, MSE recovers the DeltaNet rule, yet our chunked SwiGLU variant fails even at the training length. Across a whole chunk, the MSE residual introduces a multiplicative transition whose scale can grow with the chunk size, making the inner learning rate much harder to tune (Appendix B). The two losses guide the write differently: the dot loss rewards movement in the value direction v_t , whereas MSE follows the full residual $v_t - f_W(k_t)$ toward exact reconstruction. We tentatively hypothesize that, together with weight normalization, this simpler dot update gives LaCT a stable, well-regularized write space that outer training can organize around the hidden state, whereas MSE ties the same space more closely to visible, render-bound values.

5.3 EXPLICIT SCRATCHPAD TOKENS

The carried state does not have to live in the weights. We also test a token-space register by adding M per-layer memory tokens, which each block reads and writes through attention and carries additively across chunks as $M \leftarrow M + \Delta M$ (Fig. 3). Adding a single memory token per layer (next to the 960 video tokens of each chunk) already lifts the at-chance SWA1 baseline to perfect in-distribution accuracy and generalizes up to $N=10$ swaps. However, accuracy drops to chance by $N=20$ swaps (Fig. 6) across all token counts from 1 to 1024. We find that the plain additive update lets the carried memory norm explode on longer rollouts, while LaCT better normalizes states via fast weights and keeps extrapolating as a better-regularized register.

6 BEYOND THE CANONICAL SHELL GAME

Our Shell Game experiments show that current AR video backbones cannot compose unobserved state updates, so they should fail whenever a correct frame depends on a long swap chain among

five or more hidden objects. General video world models are much harder. Many of their prediction problems sit inside TC^0 yet remain hard to learn, while some others plausibly require NC^1 -hard state tracking and hence lie outside TC^0 if $TC^0 \neq NC^1$. More importantly, the world state is rarely a pure function of the action stream: it must also be updated from observations, whether clean context frames or the model’s own generations, and parts of it dynamically keep evolving out of sight, driven by no action at all. We close with an early exploration of two hard setups, which originally motivated this project.

Memory Maze (Pašukonis et al., 2022; Po et al., 2025; Chen et al., 2025) gives the model a clean exploration prefix and asks it to generate the frames that follow. Under local-action conditioning it receives only the discrete controls and an initial camera pose, so it has to track its pose before retrieving the visual memory tied to that location. On the original benchmark, every backbone we trained degrades sharply under this conditioning, since the MuJoCo dynamics give the same discrete action a variable displacement and pose drift accumulates (Appendix E). We rebuild the dataset with constant-displacement kinematics so that pose becomes a deterministic pure function of the actions. Tracking it is then an iterated addition of rotated unit steps in a virtually abelian subgroup of $SE(2)$, which plausibly sits in TC^0 (Merrill & Sabharwal, 2022) and is far easier than the NC^1 -complete S_5 composition. Allowing wall collisions breaks this. A blocked move becomes a no-op, so the update depends on the unobserved layout and must be corrected from what the model sees. The textured 3D Block World (Lillemark et al., 2026) tests the other case, where the blocks keep moving while the agent’s camera is turned away, so parts of the world evolve with no action at all (Appendix F).

Every backbone does well under global pose conditioning, so retrieval is not the bottleneck and what remains hard in local-action conditioning is state tracking. In most settings LaCT is one of the strongest backbones, whereas the negative-eigenvalue switch that is decisive on the Shell Game gives no consistent gain (Tabs. A4 and A5). The algebraic structure that composes an action-only permutation chain does not directly carry over, while the nonlinear fast-weight update still seems to help somewhat. Which architectures can represent these updates remains open. A real world model has to correct its state from observations, keep it fixed when an action fails, and keep it evolving even when no action arrives, and none of this follows from the actions alone.

7 CONCLUSION

We studied whether action-conditioned video diffusion models can maintain a world state that is not directly visible, using a five-cup Shell Game as a visual analogue of swap-only S_5 state tracking. Standard backbones fit the training horizon but fall toward chance as the swap chain grows, even while rendering plausible videos. The pixel-based target does not supervise that state, so the generated frames cannot carry it and the architecture has to hold it implicitly. Reliable tracking then requires a state carried across chunks, a transition expressive enough to revise it in place and realize a swap, and a gradient path from each write to its later use. Linear attention with negative transition eigenvalues and Transformers with nonlinear fast weights provide all three, and each generalizes beyond the training horizon. We further discuss Memory Maze and dynamic Block World exploration, where the world state is not determined by the action stream alone and state tracking becomes harder. Defining and learning a state that captures both the visible and hidden parts of the world remains an open challenge for video world models, and we believe this will require progress in video tokenization, stateful architectures, and objectives that reach beyond visible content.

REFERENCES

- Aditi, Niket Agarwal, Arslan Ali, Jon Allen, Martin Antolini, Adeline Aubame, Alisson G. Azolini, Junjie Bai, Maciej Bala, Yogesh Balaji, Josh Bapst, Aarti Basant, Mukesh Beladiya, Mohammad Bhat, Zaid Pervaiz Bhat, Daniel F. Blick, Vanni Brighella, Han Cai, Tiffany Cai, Eric Cameracci, Jiaxin Cao, Yulong Cao, Mark T. Carlson, Carlos Casanova, Ting-Yun Chang, Yan Chang, Yu-Wei Chao, Prithvijit Chattopadhyay, Roshan Chaudhari, Chieh yun Chen, Jun-Yu Chen, Kejiang Chen, Qizhi Chen, Wenkai Chen, Xiaotong Chen, Yu Chen, An-Chieh Cheng, C.L. Cheng, Xiu Li Chia, Jeana Choi, Chaeyeon Chung, Wenyan Cong, Yin Cui, Magdalena Dadela, Nalin Dadhich, Wenliang Dai, Joyjit Daw, Alperen Degirmenci, Rodrigo Vieira Del Monte, Robert Thomas Denommé, Sameer Dharur, Marco Di Lucca, Ke Ding, Wenhao Ding, Yifan Ding, Yuzhu Dong, Nicole Drumheller, Yilun Du, Aigul Dzhumamuratova, Aleksandr Efitorov, Hamid Eghbalzadeh, Naomi Eigbe, Imad El Hanafi, Hassan Eslami, Ben Falk, JiaoJiao Fan, Jim Fan, Amol Fasale, Sergiy Fefilatyev, Liang Feng, Francesco Ferroni, Sanja Fidler, Xiaojin Fu, Vikram Fugro, Prashant Tatyrao Gaikwad, TJ Galda, Katelyn Gao, Yihuai Gao, Wenhao Ge, Sreyan Ghosh, Arushi Goel, Vivek Goel, Akash Gokul, Rama K. Govindaraju, Jinwei Gu, Miguel Guerrero, Elfie Guo, Aryaman Gupta, Siddharth Gururani, Hugo Hadfield, Song Han, Ankur Handa, Zekun Hao, Mohammad Harrim, Ali Hassani, Nathan Hayes-Roth, Yufan He, Christopher S. Helvig, Cyrus Hogg, M.A. Huang, Michael Huang, Sophia Huang, Yufan Huang, Jacob Samuel Huffman, DeLesley S. Hutchins, Suneel Indupuru, B. Ivanovic, Arihant Jain, Joel Jang, Ryan Ji, Yanan Jian, Dongfu Jiang, Jingyi Jin, Atharva Vinay Joshi, Nikhil Yogesh Joshi, Pranjali Joshi, Jaehun Jung, Weiwei Kang, Scott Kassekert, Jan Kautz, Ashna Khetan, J Kiczka, Slawomir Kierat, Gwanghyun Kim, Kuno Kim, Sunny Kim, Kezhi Kong, Xinke Kong, Zhifeng Kong, Tomasz Kornuta, Egor Krivov, Hui Kuang, Saurav Kumar, Chia-Wen Kuo, George Kurian, Wojciech Kutak, JF Lafleche, Himangshu Lahkar, Omar Laymoun, Jayjun Lee, Sang-Gyoung Lee, Gabriele Leone, Boyi Li, Freya Li, Jiajun Li, Jinfeng Li, Lingbao Li, Pengcheng Li, Shangru Li, Ting Li, Xiaolong Li, Xuan Li, Zhao Li, Zhiqi Li, Haojie Liang, Maosheng Liao, Chenghua Lin, Tsung-Yi Lin, Mingyang Liu, Sifei Liu, Zihan Liu, Haiying Lu, Xiangyu Lu, Alice Luo, Ruipu Luo, Wenjie Luo, Jiangran Lyu, Martin Ding Ma, Nic Ma, Qianli Ma, Dawid Majchrowski, Louis-Alexandre Marcoux, Miguel Martin, Qing Miao, Ashkan Mirzaei, Shreyas Misra, Kaichun Mo, Dr. Durdana Mohsin, Hye-Jeong Moon, Pawel M. Morkisz, Saeid Motiian, Kirill Motkov, Seungjun Nah, Yashraj S. Narang, Deepak Narayanan, Thabang Ngazimbi, Julian Ouyang, David Page, Yatian Pang, Sehwi Park, Mahesh Kishore Patekar, Mostofa Patwary, Marco Pavone, Trung Xuan Pham, Wei Ping, Soha Pouya, Shrimai Prabhumoye, Varun Praveen, Delin Qu, Hesam Rabeti, Morteza Ramezanali, Margaret Ingeborg Reeb, Xuanchi Ren, Kristen Rumley, Wojciech Rymer, Jun Saito, Yeongho Seol, John Shao, Piyush Shekdar, Tianwei Shen, Humphrey Shi, Min Shi, Stella Shi, Kevin J. Shih, Mohammad Shoeibi, Mateusz Sieniawski, Shuran Song, Alexander Sotelo, Amir Mojtaba Sotoodeh, Sunil Srinivasa, Vignesh Srinivasakumar, Bartosz Stefaniak, Rahul Heinrich Steiger, Shangkun Sun, Jiaxiang Tang, Shitao Tang, Yan Tang, Yue Tang, Tolou Tavakkoli, Kayley Ting, Krzysztof Tomala, Wei-Cheng Tseng, Jibin Yesudas Varghese, S F Vasil'ev, Thomas Volk, Raju Wagwani, Roger Waleffe, Andrew Z. Wang, Boxian Wang, Haoxiang Wang, Qiaolin Wang, Shihao Wang, Shijie Wang, Tingchang Wang, Yan Wang, Yu Wang, David Wehr, Fangyin Wei, Xinshuo Weng, Jay Zhangjie Wu, Kedi Wu, Hongchi Xia, Summer Xiao, Tianjun Xiao, Kevin Xie, Daguang Xu, Jiashu Xu, Mengya Xu, Ruqing Xu, Xingqian Xu, Yao Xu, Dinghao Yang, Dong Yang, Han Yang, Xiaodong Yang, Xuning Yang, Yichu Yang, Yurong You, Zhiding Yu, Haobo Yuan, S. Chung Kim Yuen, Xiaohui Zeng, Pengcuo Zeren, Cindy Zha, Haotian Zhang, Jenny Zhang, Jing Zhang, Liang Zhang, Paris Zhang, Shun Zhang, Xuan Zhang, Zhizheng Zhang, Annabeth Xinyu Zhao, Yilin Zhao, Yuliya Zhautouskaya, Charles Zhou, Fengzhe Zhou, Shilin Zhu, Yuke Zhu, Dima Zhytko, and Artur Zolkowski. *Cosmos 3: Omnimodal world models for physical ai*. 2026. URL <https://api.semanticscholar.org/CorpusID:288863612>.
- Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, JiaoJiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Samuel Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, G. Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixé, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezanali, Fitsum A. Reda,

- Xiao-Shuai Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne P. Tchapmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Rajan Varghese, Hao Wang, Haoxiang Wang, Hengyi Wang, Tingwei Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yuan Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zolkowski. Cosmos world foundation model platform for physical ai. *ArXiv*, abs/2501.03575, 2025. URL <https://api.semanticscholar.org/CorpusID:275342594>.
- Luca Savant Aira, Antonio Montanaro, Emanuele Aiello, Diego Valsesia, and Enrico Magli. Motioncraft: Physics-based zero-shot video generation. *ArXiv*, abs/2405.13557, 2024. URL <https://api.semanticscholar.org/CorpusID:269982890>.
- David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in nc1. In *Symposium on the Theory of Computing*, 1986. URL <https://api.semanticscholar.org/CorpusID:9031549>.
- Nvidia Aarti Basant, Amlan Kar, Despoina Paschalidou, Fangyin Wei, Francesco Ferroni, Guillermo Garcia Cobo, Haithem Turki, Huan Ling, Jae-Bok Seo, James Lucas, Jay Zhangjie Wu, Jialiang Wang, Jonathan Lorraine, Junyu Gao, Kai He, Katar'ina T'othov'a, Kevin Xie, Michał Tyszkiewicz, Qi Wu, Riccardo de Lutio, Ruilong Li, Sanja Fidler, Seung Wook Kim, Tianchang Shen, Tianshi Cao, Tobias Pfaff, William Lew, Xindi Wu, Xuanchi Ren, Yifan Lu, Yuxuan Zhang, Zan Gojcic, and Zian Wang. Nvidia omnidreams: Real-time generative world model for closed-loop autonomous vehicle simulation. 2026. URL <https://api.semanticscholar.org/CorpusID:288863182>.
- Ali Behrouz, Peilin Zhong, and Vahab S. Mirrokni. Titans: Learning to memorize at test time. *ArXiv*, abs/2501.00663, 2024. URL <https://api.semanticscholar.org/CorpusID:275212078>.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal M. P. Behbahani, Stephanie Chan, Nicolas Manfred Otto Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott E. Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktaschel. Genie: Generative interactive environments. *ArXiv*, abs/2402.15391, 2024. URL <https://api.semanticscholar.org/CorpusID:267897982>.
- Jorge Diaz Chao, Konpat Preechakul, Yuxi Liu, and Yutong Bai. The seriality gap in video diffusion models. 2026. URL <https://api.semanticscholar.org/CorpusID:290178331>.
- Boyuan Chen, Diego Marti Monso, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *ArXiv*, abs/2407.01392, 2024. URL <https://api.semanticscholar.org/CorpusID:270869622>.
- Taiye Chen, Zihan Ding, Anjian Li, Christina Zhang, Zeqi Xiao, Yisen Wang, and Chi Jin. Recurrent autoregressive diffusion: Global memory meets local attention. *ArXiv*, abs/2511.12940, 2025. URL <https://api.semanticscholar.org/CorpusID:283072802>.
- Karan Dalal, Daniel Kocaja, Gashon Hussein, Jiarui Xu, Yue Zhao, Youjin Song, Shihao Han, Ka Chun Cheung, Jan Kautz, Carlos Guestrin, Tatsunori Hashimoto, Sanmi Koyejo, Yejin Choi, Yu Sun, and Xiaolong Wang. One-minute video generation with test-time training. 2025 *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 17702–17711, 2025. URL <https://api.semanticscholar.org/CorpusID:277627093>.
- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *ArXiv*, abs/2405.21060, 2024. URL <https://api.semanticscholar.org/CorpusID:270199762>.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. *ArXiv*, abs/1807.03819, 2018. URL <https://api.semanticscholar.org/CorpusID:49667762>.

- Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flex attention: A programming model for generating optimized attention kernels. *ArXiv*, abs/2412.05496, 2024. URL <https://api.semanticscholar.org/CorpusID:274598006>.
- Zicheng Duan, Jiatong Xia, Zeyu Zhang, Wenbo Zhang, G. Zhou, Chenhui Gou, Yefei He, Feng Chen, Xinyu Zhang, and Lingqiao Liu. Liveworld: Simulating out-of-sight dynamics in generative video world models. 2026. URL <https://api.semanticscholar.org/CorpusID:286371918>.
- Shenyuan Gao, William Liang, Kaiyuan Zheng, Aravind Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, Qianli Ma, Seungjun Nah, Loic Magne, Jiannan Xiang, Yuqi Xie, Ruijie Zheng, Dantong Niu, You Liang Tan, K. R. Zentner, George Kurian, Suneel Indupuru, Pooya Jannaty, Jinwei Gu, Jun Zhang, Jitendra Malik, Pieter Abbeel, Ming-Yu Liu, Yuke Zhu, Joel Jang, and LinxiJimFan. Dreamdojo: A generalist robot world model from large-scale human videos. *ArXiv*, abs/2602.06949, 2026. URL <https://api.semanticscholar.org/CorpusID:285401818>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kaillkhura, Abhinav Bhatle, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *ArXiv*, abs/2502.05171, 2025. URL <https://api.semanticscholar.org/CorpusID:276235803>.
- Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. *ArXiv*, abs/2301.13196, 2023. URL <https://api.semanticscholar.org/CorpusID:256389656>.
- Nate Gillman, Charles Herrmann, Michael Freeman, Daksh Aggarwal, Evan Luo, Deqing Sun, and Chen Sun. Force prompting: Video generation models can learn and generalize physics-based control signals. *ArXiv*, abs/2505.19386, 2025. URL <https://api.semanticscholar.org/CorpusID:278905832>.
- Riccardo Grazi, Julien N. Siems, Jorg K. H. Franke, Arber Zela, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *ArXiv*, abs/2411.12537, 2024. URL <https://api.semanticscholar.org/CorpusID:274141450>.
- Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated dltanet-2: Decoupling erase and write in linear attention. 2026. URL <https://api.semanticscholar.org/CorpusID:288649267>.
- Xianglong He, Chunli Peng, Zexiang Liu, Boyang Wang, Yifan Zhang, Qiushi Cui, Fei Kang, Biao Jiang, Mengyin An, Yang Ren, Baixin Xu, Hao Guo, Kaixiong Gong, Cyrus Wu, Wei Li, Xuchen Song, Yang Liu, Eric Li, and Yahui Zhou. Matrix-game 2.0: An open-source, real-time, and streaming interactive world model. *ArXiv*, abs/2508.13009, 2025. URL <https://api.semanticscholar.org/CorpusID:280677722>.
- Jonathan Ho, Ajay Jain, and P. Abbeel. Denoising diffusion probabilistic models. *ArXiv*, abs/2006.11239, 2020. URL <https://api.semanticscholar.org/CorpusID:219955663>.
- Yicong Hong, Yiqun Mei, Chongjian Ge, Yiran Xu, Yang Zhou, Sai Bi, Yannick Hold-Geoffroy, Mike Roberts, Matthew Fisher, Eli Shechtman, Kalyan Sunkavalli, Feng Liu, Zhengqi Li, and Hao Tan. Relic: Interactive video world model with long-horizon memory. *ArXiv*, abs/2512.04040, 2025. URL <https://api.semanticscholar.org/CorpusID:283467075>.
- Tianyu Huang, Wangguandong Zheng, Tengfei Wang, Yuhao Liu, Zhenwei Wang, Junta Wu, Jie Jiang, Hui Li, Rynson W. H. Lau, Wangmeng Zuo, and Chunchao Guo. Voyager: Long-range and world-consistent video diffusion for explorable 3d scene generation. *ACM Transactions on Graphics (TOG)*, 44:1 – 15, 2025a. URL <https://api.semanticscholar.org/CorpusID:279155015>.

- Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *ArXiv*, abs/2506.08009, 2025b. URL <https://api.semanticscholar.org/CorpusID:279251392>.
- Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL <https://arxiv.org/abs/2510.04871>.
- Bingyi Kang, Yang Yue, Rui Lu, Zhijie Lin, Yang Zhao, Kaixin Wang, Gao Huang, and Jiashi Feng. How far is video generation from world model: A physical law perspective. *ArXiv*, abs/2411.02385, 2024. URL <https://api.semanticscholar.org/CorpusID:273822016>.
- Yekun Ke, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Circuit complexity bounds for visual autoregressive model. *ArXiv*, abs/2501.04299, 2025. URL <https://api.semanticscholar.org/CorpusID:275358227>.
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *ArXiv*, abs/2601.16163, 2026. URL <https://api.semanticscholar.org/CorpusID:284935587>.
- Kunyang Li, Mubarak Shah, and Yuzhang Shang. Attend locally, remember linearly: Linear attention as cross-frame memory for autoregressive video diffusion. 2026. URL <https://api.semanticscholar.org/CorpusID:288655935>.
- Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. Normuon: Making muon more efficient and scalable. *ArXiv*, abs/2510.05491, 2025. URL <https://api.semanticscholar.org/CorpusID:281886129>.
- Hansen Jin Lillemark, Benhao Huang, Fangneng Zhan, Yilun Du, and Thomas Anderson Keller. Flow equivariant world models: Memory for partially observed dynamic environments. *ArXiv*, abs/2601.01075, 2026. URL <https://api.semanticscholar.org/CorpusID:284487081>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *ArXiv*, abs/2210.02747, 2022. URL <https://api.semanticscholar.org/CorpusID:252734897>.
- Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. *ArXiv*, abs/2210.10749, 2022a. URL <https://api.semanticscholar.org/CorpusID:252992725>.
- Fangfu Liu, Kaiming He, Tianchang Shen, Tianshi Cao, Sanja Fidler, Yueqi Duan, Jun Gao, Igor Gilitschenski, Zian Wang, and Xuanchi Ren. Gamma-world: Generative multi-agent world modeling beyond two players. 2026a. URL <https://api.semanticscholar.org/CorpusID:288702163>.
- Juncheng Liu, Sven Elfle, Or Litany, Zan Gojcic, and Ruilong Li. Test-time training with kv binding is secretly linear attention. *ArXiv*, abs/2602.21204, 2026b. URL <https://api.semanticscholar.org/CorpusID:286001329>.
- Tiedong Liu and Wee Sun Lee. Can vision-language models solve the shell game? 2026. URL <https://api.semanticscholar.org/CorpusID:286377341>.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *ArXiv*, abs/2209.03003, 2022b. URL <https://api.semanticscholar.org/CorpusID:252111177>.
- Yuxi Liu, Konpat Preechakul, Kananart Kuwarananchaoen, and Yutong Bai. The serial scaling hypothesis. *ArXiv*, abs/2507.12549, 2025. URL <https://api.semanticscholar.org/CorpusID:280265538>.

- Ziqi Ma, Mengzhan Liufu, and Georgia Gkioxari. Out of sight, out of mind? evaluating state evolution in video world models. 2026. URL <https://api.semanticscholar.org/CorpusID:286520075>.
- William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2022. URL <https://api.semanticscholar.org/CorpusID:257050206>.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. *ArXiv*, abs/2310.07923, 2024. URL <https://api.semanticscholar.org/CorpusID:263909434>.
- William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models. *ArXiv*, abs/2404.08819, 2024. URL <https://api.semanticscholar.org/CorpusID:269149086>.
- William Merrill, Hongjian Jiang, Yanhong Li, Anthony Widjaja Lin, and Ashish Sabharwal. Why are linear rnns more parallelizable? 2026. URL <https://api.semanticscholar.org/CorpusID:286238437>.
- Mayank Mishra, Shawn Tan, Ion Stoica, Joseph Gonzalez, and Tri Dao. M²RNN: Non-linear rnns with matrix-valued states for scalable language modeling. 2026. URL <https://api.semanticscholar.org/CorpusID:286572414>.
- Jisu Nam, Yicong Hong, Chun-Hao Paul Huang, Feng Liu, Joungbin Lee, Jiyoung Kim, Siyoon Jin, Yunsung Lee, Jaeyoon Jung, Suhwan Choi, Seungryong Kim, and Yang Zhou. Worldcam: Interactive autoregressive 3d gaming worlds with camera pose as a unifying geometric representation. 2026. URL <https://api.semanticscholar.org/CorpusID:286584040>.
- Jack Parker-Holder, Philip Ball, Jake Bruce, Vibhavari Dasagi, Kristian Holsheimer, Christos Kaplanis, Alexandre Moufaret, Guy Scully, Jeremy Shar, Jimmy Shi, Stephen Spencer, Jessica Yung, Michael Dennis, Sultan Kenjeyev, Shangbang Long, Vlad Mnih, Harris Chan, Maxime Gazeau, Bonnie Li, Fabio Pardo, Luyu Wang, Lei Zhang, Frederic Besse, Tim Harley, Anna Mitenkova, Jane Wang, Jeff Clune, Demis Hassabis, Raia Hadsell, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 2: A large-scale foundation world model. 2024. URL <https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/>.
- J. Pašukonis, Timothy P. Lillicrap, and Danijar Hafner. Evaluating long-term memory in 3d mazes. *ArXiv*, abs/2210.13383, 2022. URL <https://api.semanticscholar.org/CorpusID:253097769>.
- William S. Peebles and Saining Xie. Scalable diffusion models with transformers. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4172–4182, 2022. URL <https://api.semanticscholar.org/CorpusID:254854389>.
- Ryan Po, Yotam Nitzan, Richard Zhang, Berlin Chen, Tri Dao, Eli Shechtman, Gordon Wetstein, and Xun Huang. Long-context state-space video world models. *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 8733–8744, 2025. URL <https://api.semanticscholar.org/CorpusID:278911218>.
- Xuanchi Ren, Yifan Lu, Tianshi Cao, Ruiyuan Gao, Shengyu Huang, Amirmojtaba Sabour, Tianchang Shen, Tobias Pfaff, Jay Zhangjie Wu, Runjian Chen, Seung Wook Kim, Jun Gao, Laura Leal-Taixé, Mike Chen, Sanja Fidler, and Huan Ling. Cosmos-drive-dreams: Scalable synthetic driving data generation with world foundation models. *ArXiv*, abs/2506.09042, 2025a. URL <https://api.semanticscholar.org/CorpusID:279261009>.
- Xuanchi Ren, Tianchang Shen, Jiahui Huang, Huan Ling, Yifan Lu, Merlin Nimier-David, Thomas Muller, Alexander Keller, Sanja Fidler, and Jun Gao. Gen3c: 3d-informed world-consistent video generation with precise camera control. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6121–6132, 2025b. URL <https://api.semanticscholar.org/CorpusID:276782107>.

- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. *ArXiv*, abs/2502.17416, 2025. URL <https://api.semanticscholar.org/CorpusID:276575815>.
- Nedko Savov, Naser Kazemi, Deheng Zhang, Danda Pani Paudel, Xi Wang, and Luc van Gool. Statespacediffuser: Bringing long context to diffusion world models. *ArXiv*, abs/2505.22246, 2025. URL <https://api.semanticscholar.org/CorpusID:278959395>.
- George Savva, Oscar Michel, Daohan Lu, Suppakit Waiwitlikhit, Timothy Patrick Meehan, Dr. Renu Mishra, Srivats Poddar, Jack Lu, and Saining Xie. Solaris: Building a multi-player video world model in minecraft. *ArXiv*, abs/2602.22208, 2026. URL <https://api.semanticscholar.org/CorpusID:286011266>.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021. URL <https://api.semanticscholar.org/CorpusID:235377069>.
- Mark Schöne, Babak Rahmani, Heiner Kremer, Fabian Falck, Hitesh Ballani, and Jannes Gladrow. Implicit language models are rnns: Balancing parallelization and expressivity. *ArXiv*, abs/2502.07827, 2025. URL <https://api.semanticscholar.org/CorpusID:276287340>.
- Junyoung Seo, Hyunwook Choi, Min-Joon Kwon, Jinhyeok Choi, Siyoon Jin, Gayoung Lee, Junho Kim, Joungbin Lee, Geonmo Gu, Dongyoon Han, Sangdoo Yun, Seungryong Kim, and Jin-Hwa Kim. Grounding world simulation models in a real-world metropolis. 2026. URL <https://api.semanticscholar.org/CorpusID:286572548>.
- Joonghyuk Shin, Zhengqi Li, Richard Zhang, Jun-Yan Zhu, Jaesik Park, E. Shechtman, and Xun Huang. Motionstream: Real-time video generation with interactive motion controls. *ArXiv*, abs/2511.01266, 2025. URL <https://api.semanticscholar.org/CorpusID:282739800>.
- Julien N. Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. Deltaproduct: Improving state-tracking in linear rnns via householder products. 2025. URL <https://api.semanticscholar.org/CorpusID:277434955>.
- Julien N. Siems, Riccardo Grazi, Kirill P. Kalinin, Hitesh Ballani, and Babak Rahmani. Learning state-tracking from code using linear rnns. *ArXiv*, abs/2602.14814, 2026. URL <https://api.semanticscholar.org/CorpusID:285616800>.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. *ArXiv*, abs/2303.01469, 2023. URL <https://api.semanticscholar.org/CorpusID:257280191>.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Ge Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Oluwasanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. *ArXiv*, abs/2407.04620, 2024. URL <https://api.semanticscholar.org/CorpusID:271039606>.
- Arnav Tandon, Karan Dalal, Xinhao Li, Daniel Kocaja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context. *ArXiv*, abs/2512.23675, 2025. URL <https://api.semanticscholar.org/CorpusID:284313495>.
- Junshu Tang, Jiacheng Liu, Jiaqi Li, Longhuang Wu, Haoyu Yang, Penghao Zhao, Siruis Gong, Xiang Yuan, Shuai Shao, and Qinglin Lu. Hunyuan-gamecraft-2: Instruction-following interactive game world model. *ArXiv*, abs/2511.23429, 2025. URL <https://api.semanticscholar.org/CorpusID:283438286>.
- Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Xiaofeng Meng, Ningying Zhang,

- Pandeng Li, Ping Wu, Ruihang Chu, Rui Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wen-Chao Zhou, Wenten Wang, Wen Shen, Wenyuan Yu, Xianzhong Shi, Xiaomin Huang, Xin Xu, Yan Kou, Yan-Mei Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhigang Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models. *ArXiv*, abs/2503.20314, 2025a. URL <https://api.semanticscholar.org/CorpusID:277321639>.
- Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Chang-Le Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi-Yadkori. Hierarchical reasoning model. *ArXiv*, abs/2506.21734, 2025b. URL <https://api.semanticscholar.org/CorpusID:280012234>.
- Zile Wang, Zexiang Liu, Jaixing Li, Kaichen Huang, Baixin Xu, Fei Kang, Mengyin An, Peiyu Wang, Biao Jiang, Yichen Wei, Yidan Xietian, Jiangbo Pei, Liang Hu, Bo Jiang, Hua Xue, Zidong Wang, Hao Sun, Wei Li, Wanli Ouyang, Xianglong He, Yang Liu, Yangguang Li, and Yahui Zhou. Matrix-game 3.0: Real-time and streaming interactive world model with long-horizon memory. 2026. URL <https://api.semanticscholar.org/CorpusID:287351855>.
- Haoyu Wu, Diankun Wu, Tianyu He, Junliang Guo, Yang Ye, Yueqi Duan, and Jiang Bian. Geometry forcing: Marrying video diffusion and 3d representation for consistent world modeling. *ArXiv*, abs/2507.07982, 2025a. URL <https://api.semanticscholar.org/CorpusID:280234402>.
- Tong Wu, Shuai Yang, Ryan Po, Yinghao Xu, Ziwei Liu, Dahua Lin, and Gordon Wetzstein. Video world models with long-term spatial memory. *ArXiv*, abs/2506.05284, 2025b. URL <https://api.semanticscholar.org/CorpusID:279244178>.
- Zeqi Xiao, Yushi Lan, Yifan Zhou, Wenqi Ouyang, Shuai Yang, Yanhong Zeng, and Xingang Pan. Worldmem: Long-term consistent world simulation with memory. *ArXiv*, abs/2504.12369, 2025. URL <https://api.semanticscholar.org/CorpusID:277857150>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *ArXiv*, abs/2412.06464, 2024a. URL <https://api.semanticscholar.org/CorpusID:274598177>.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *ArXiv*, abs/2406.06484, 2024b. URL <https://api.semanticscholar.org/CorpusID:270371554>.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Aravind Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan C. Julian, Dan Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, LinxiJimFan, and Joel Jang. World action models are zero-shot policies. *ArXiv*, abs/2602.15922, 2026. URL <https://api.semanticscholar.org/CorpusID:285726165>.
- Jung Yi, Minjae Kim, Paul Hyunbin Cho, Wooseok Jang, Sangdoo Yun, and Seungryong Kim. Worldkv: Efficient world memory with world retrieval and compression. *ArXiv*, abs/2605.22718, 2026. URL <https://api.semanticscholar.org/CorpusID:288650256>.
- Tianwei Yin, Michael Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6613–6623, 2023. URL <https://api.semanticscholar.org/CorpusID:265506768>.
- Tianwei Yin, Qiang Zhang, Richard Zhang, William T. Freeman, Frédo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22963–22974, 2024. URL <https://api.semanticscholar.org/CorpusID:274610175>.

- Sihyun Yu, Nanye Ma, Pinzhi Huang, Hyunseok Lee, Shusheng Yang, June Suk Choi, Ellis Brown, Oscar Michel, Boyang Zheng, Jinwoo Shin, and Saining Xie. Benchmarking visual state tracking in multimodal video understanding. 2026. URL <https://api.semanticscholar.org/CorpusID:288903827>.
- Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right. *ArXiv*, abs/2505.23884, 2025a. URL <https://api.semanticscholar.org/CorpusID:279071244>.
- Yifan Zhang, Chunli Peng, Boyang Wang, Puyi Wang, Qingchen Zhu, Fei Kang, Biao Jiang, Zedong Gao, Eric Li, Yang Liu, and Yahui Zhou. Matrix-game: Interactive world foundation model. *ArXiv*, abs/2506.18701, 2025b. URL <https://api.semanticscholar.org/CorpusID:280000146>.
- Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Long Yu, Ye-Jia Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianling Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bo Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Hua Zheng, Feng Wang, Yibo Liu, Meng xiao Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Ji-Hua Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyue Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture. *ArXiv*, abs/2510.26692, 2025c. URL <https://api.semanticscholar.org/CorpusID:282592221>.
- Yuyang Zhao, Yicheng Pan, Qiyuan He, Jincheng Yu, Junsong Chen, Tian Ye, Haozhe Liu, Enze Xie, and Song Han. Sana-streaming: Real-time streaming video editing with hybrid diffusion transformer. 2026. URL <https://api.semanticscholar.org/CorpusID:288814323>.
- Haoyi Zhu, Haozhe Liu, Yuyang Zhao, Tian Ye, Junsong Chen, Jincheng Yu, Tong He, Song Han, and Enze Xie. Sana-wm: Efficient minute-scale world modeling with hybrid linear diffusion transformer. 2026. URL <https://api.semanticscholar.org/CorpusID:288254740>.

A AUTOREGRESSIVE ROLLOUT ALONE DOES NOT TRAIN STATE UPDATES

A chunk-wise autoregressive DiT generates chunks sequentially, but this ordering by itself does not say what state passes between calls or teach the model what to write. Language models can use their output tokens as an append-only scratchpad, with each new token storing the latest state for the next decoding step (Merrill & Sabharwal, 2024) and the KV cache makes this trace more efficient. An AR video DiT similarly has two possible carriers, the KV cache and the generated video tokens.

The KV cache cannot update state in place. A KV entry at layer ℓ is computed from that layer’s input, so reading and updating it can only write the new state at a higher layer, where it becomes readable. In the Shell Game, the initial reveal places the ball position in the cache at every layer. The first swap can read it at layer 1 and write the updated arrangement at layer 2. The next swap reads the latest arrangement at layer 2 and writes its update at layer 3. Each swap therefore pushes the state upward until it reaches the top of the network. An internal recurrence (*e.g.*, RNNs, linear attentions and TTT) avoids this constraint by applying each transition to the previous state within the same layer.

Generated chunks are not trained as state tokens. At inference, a generated video chunk returns as input to the next pass and could in principle carry the updated arrangement like chain-of-thought (CoT) tokens. In our training setup, however, the frozen Wan VAE maps each teacher chunk to a visual latent (Wang et al., 2025a). For an occluded swap, this target represents the rendered cup motion but contains no label for the hidden ball position. Teacher forcing and diffusion forcing provide the next chunk with this teacher latent rather than a model-written output (Chen et al., 2024). A later reveal error therefore cannot teach an earlier generated chunk what to preserve through its effect on the remaining rollout. The denoising loop repeats the backbone within each chunk, but every call is trained toward the same target and the number of calls does not grow with the swap chain. This is consistent with our denoising-step sweep and the analysis of Liu et al. (2025).

Self Forcing feeds generated chunks back during training (Huang et al., 2025b), but its practical implementation passes gradients through one selected denoising call per chunk and detaches earlier cached representations. Later errors therefore still cannot train earlier chunks through their down-stream effects. Full rollout backpropagation, explicit state targets, or task-specific rewards could provide this supervision, though we do not test them here as they could be too compute heavy or less generic. The recurrent variants instead maintain an internal state whose write and later use remain connected during training.

B LINEAR TTT UPDATES AS LINEAR ATTENTION AND DELTANET

For a bias-free linear inner model, a single test-time-training (TTT) gradient step yields either an additive linear-attention update or the DeltaNet rule, depending on the inner loss (Sun et al., 2024; Liu et al., 2026b).

Setup. For $t = 1, 2, \dots$, a token x_t is projected by slow (outer-loop) weights to a query and key $q_t, k_t \in \mathbb{R}^{d_k}$ and a value $v_t \in \mathbb{R}^{d_v}$. The fast state is $S_t \in \mathbb{R}^{d_k \times d_v}$, the linear inner model is $f_{S_t}(k) = S_t^\top k$, and the query readout is $o_t = S_t^\top q_t \in \mathbb{R}^{d_v}$. A TTT layer updates S by one gradient step on an inner loss ℓ_t evaluated at the current key–value pair, $S_t = S_{t-1} - \eta_t \nabla_S \ell_t(S_{t-1})$.

Plain linear attention from the dot-product KV-binding loss. For the negative dot-product KV-binding objective $\ell_t(S) = -\langle S^\top k_t, v_t \rangle = -k_t^\top S v_t$, the gradient can be calculated as $\nabla_S \ell_t = -k_t v_t^\top$. Taking a single update step yields

$$S_t = S_{t-1} + \eta_t k_t v_t^\top, \quad o_t = S_t^\top q_t = S_0^\top q_t + \sum_{i=1}^t \eta_i v_i (k_i^\top q_t), \quad (\text{A1})$$

where S_0 is the state before the first update. Setting $S_0 = 0$ gives the additive outer-product memory of causal linear attention (Schlag et al., 2021), and Liu et al. (2026b) showed that this could be equivalent to TTT-KVB. Summing the per-step updates also reproduces the batch gradient-descent form of Sun et al. (2024, Thm. 1).

DeltaNet from the MSE KV-binding loss. Now take the MSE objective $\ell_t(S) = \frac{1}{2}\|S^\top k_t - v_t\|^2$, whose gradient is $\nabla_S \ell_t = k_t(S^\top k_t - v_t)^\top$. Taking one step here gives

$$S_t = S_{t-1} - \eta_t k_t (S_{t-1}^\top k_t - v_t)^\top = (I - \eta_t k_t k_t^\top) S_{t-1} + \eta_t k_t v_t^\top. \quad (\text{A2})$$

Evaluating the updated state at the current key makes the overwrite explicit. For $\|k_t\|_2 = 1$, $S_t^\top k_t = (1 - \eta_t) S_{t-1}^\top k_t + \eta_t v_t$, so η_t interpolates between the previous value associated with k_t and the new value v_t . At $\eta_t = 1$, the update replaces the previous value exactly. This is the DeltaNet delta rule with write strength $\beta_t = \eta_t$ (Yang et al., 2024b), so online TTT-Linear with the MSE loss recovers DeltaNet (Sun et al., 2024).

Chunked updates and learning-rate scaling. For a chunk $\mathcal{C}_m$ of size c , collect the keys and values as $K_m = [k_i]_{i \in \mathcal{C}_m}$ and $V_m = [v_i]_{i \in \mathcal{C}_m}$, and let $D_m = \text{diag}(\eta_i) \succeq 0$ contain their learning rates. A single gradient step on the chunk’s summed loss, with all per-token gradients taken at the pre-update state S_{m-1} gives

$$S_m^{\text{dot}} = S_{m-1} + K_m D_m V_m^\top, \quad S_m^{\text{mse}} = (I - K_m D_m K_m^\top) S_{m-1} + K_m D_m V_m^\top. \quad (\text{A3})$$

For $c = 1$, the MSE rule is exactly DeltaNet. For $c > 1$, it is a single batched-gradient step rather than a product of sequential DeltaNet updates. Writing $G_m = K_m D_m K_m^\top$, the raw MSE transition is non-expansive in Frobenius norm exactly when the largest eigenvalue satisfies $\lambda_{\max}(G_m) \leq 2$. Increasing the chunk size adds positive-semidefinite rank-one terms to G_m and can therefore make the raw transition expansive, especially when the features are correlated. Keeping the update non-expansive then requires shrinking the inner learning rate with the chunk size, down to $\eta \leq 2/c$ in the worst case. For example, in our setting a single update aggregates the $c = 960$ tokens of a video chunk. The dot rule has no corresponding multiplicative transition in this linear setting.

For the SwiGLU inner model, with the feature map $\phi(x) = \text{silu}(W_0 x) \odot (W_2 x)$ and readout matrix W_1 of Sec. 5.2, the update of W_1 takes exactly this form with K_m replaced by $\Phi_m = [\phi(k_i)]_{i \in \mathcal{C}_m}$. Freezing the feature map therefore leaves chunked linear attention under a fixed kernel, which is the variant that fails past the training length in Tab. 1, while full LaCT also updates W_0 and W_2 and restores the row norms of all three fast matrices after each chunk.

Connection to the Shell Game inner-loss ablation. In our chunked SwiGLU ablation, the dot-product objective preserves state tracking, whereas MSE fails even at the training horizon (Tab. 1). We hypothesize that the dot-product objective works better in our setting because it provides a less restrictive binding signal, rewarding a larger inner product between $f_W(k)$ and v without requiring an exact match. MSE instead drives $f_W(k)$ toward exact reconstruction of the projected visual value v , which we believe leaves less freedom for the fast weights to encode unobserved state not specified by the current visual target. As shown above for a simple linear inner model, the MSE residual also introduces a state-dependent multiplicative transition that can become unstable as the chunk grows. Even under the dot-product objective, full LaCT does not reduce to a static-kernel prefix sum, since updating W_0 and W_2 online makes its nonlinear feature map history-dependent (Liu et al., 2026b).

C TOY STATE-TRACKING EXPERIMENT DETAILS

Task. We use the symmetric-group word problems S_3 and S_5 in the swap-only setting (Grazzi et al., 2024; Yang et al., 2024b). Each action is a transposition of two elements, and the target at every step is the running prefix product of the actions so far. The model receives a single initial state and the action stream and must predict the current permutation at each step, with no intermediate supervision of the hidden state.

Models and Depth Sweep. We compare nonlinear RNNs (RNN, GRU, LSTM), a Transformer, Mamba2, Gated DeltaNet (GDN), and Kimi Delta Attention (KDA), together with GDN-neg and KDA-neg, which widen the transition spectrum from $[0, 1]$ to $[-1, 1]$. Each model receives the initial permutation once and predicts the full running permutation after every swap. The RNN, GRU, and LSTM initialize their recurrent state from the initial permutation, while the remaining backbones prepend it as an input token. To approximately match the parameter count per layer across families, we use a fixed width within each family throughout the sweep, with widths of 256

for the Transformer, 288 for Mamba2, 304 for GDN and KDA, and 644, 376, and 324 for the RNN, GRU, and LSTM. The Transformer uses four attention heads, Mamba2 uses a state dimension of 16, a convolution width of 4, and an expansion ratio of 2, and GDN and KDA use four recurrent heads.

All models are trained from scratch on sequences of $N = 128$ swaps for up to 300K steps using AdamW with a batch size of 256 and a learning rate of 10^{-4} . We define a model as solving a length when its best validation checkpoint reaches a final-step accuracy of 0.95, against chance levels of $1/6$ for S_3 and $1/120$ for S_5 . Fig. 1 first evaluates the Transformer depth sweep at lengths $N \in \{8, 16, 32, 64, 128\}$ and reports the minimum depth that solves each length. It then applies the same criterion across architectures at $N = 128$, starting from one layer and increasing over the tested even depths until the threshold is reached. For the extrapolation results, we take the minimum-depth model that solves $N = 128$ for each architecture and evaluate it at $N \in \{128, 256, 512, 1000\}$. Since the RNN, GRU, and LSTM all solve both tasks with a single layer, we show the vanilla RNN as their representative in the figure.

D THE VISUAL SHELL GAME EXPERIMENT DETAILS

We adapt the cups-and-ball renderer of Liu & Lee (2026), implemented in Three.js and WebGL, and add explicit lift and reveal phases together with action annotations. Videos are rendered at 256×192 resolution. For k cups, the action vocabulary contains $\binom{k}{2}$ pairwise swap actions and three global controls for lifting all cups, lowering them, or no-op leaving the scene unchanged. With $k = 5$, this gives 10 swap actions and 13 actions in total.

An episode begins by lifting all cups to reveal the initial ball position and lowering them again. It then applies N pairwise swaps while the ball remains occluded and ends with a final reveal. We use autoregressive chunks of 5 latent frames, corresponding to approximately 20 pixel frames, and align the opening reveal, lower, each swap, and final reveal with one chunk. Thus, an episode with N swaps contains $N + 3$ chunks and $5(N + 3)$ latent frames. The Wan 2.1 VAE maps the first latent to one pixel frame and each subsequent latent to four, giving

$$4 \cdot 5(N + 3) - 3 = 20N + 57$$

pixel frames. Using this equation, $N = 5$ produces 8 chunks, 40 latent frames, and 157 pixel frames, while $N = 30$ produces 33 chunks, 165 latent frames, and 657 pixel frames. We train on $N = 5$ swaps and evaluate up to $N = 30$. With five initial ball positions and 10 choices per swap, there are 5×10^N possible episodes of length N , making memorization infeasible and length extrapolation the primary diagnostic.

D.1 MODEL ARCHITECTURE

All variants share a Wan-style latent video diffusion scaffold (Wang et al., 2025a). The frozen Wan2.1 VAE maps each 256×192 video to 16-channel latents with $4 \times$ temporal and $8 \times$ spatial compression. Each autoregressive chunk contains five latent frames on a 32×24 grid, and $1 \times 2 \times 2$ patches produce 960 tokens per chunk.

We set the model width to $D = 1152$ with 16 attention heads of dimension 72, QK normalization, 3D RoPE, and a two-layer GELU FFN of width $4D$. For conditioning, we project a 256-dimensional sinusoidal timestep embedding and a 72-dimensional lookup for the 13 actions to D and inject both through AdaLN. The pure-attention and scratchpad models use 12 layers, while the recurrent families adjust their depth to remain at a broadly comparable scale around 200M parameters. Causal models use SWA1 by default, attending to the current and previous chunks.

Tab. A1 reports the layer count, internal mixer geometry, recurrent transition range, and exact parameter count for the standard and linear-attention backbones. The RNN, Mamba2, and linear-attention families use eight layers with temporal subchunk sizes $[1, 1, 1, 1, 2, 2, 4, 4]$ from bottom to top. All linear-attention rows use 12 recurrent heads of width 96. The negative-eigenvalue variants change only the transition parameterization from $[0, 1]$ to $[-1, 1]$.

LaCT and the explicit scratchpad carry persistent per-layer state across chunks, with their configurations summarized in Tab. A2. For LaCT with model width D , H fast-weight heads, and SwiGLU expansion $r = 2$, each head has width $d = D/H$, and the three fast matrices carry $3rD^2/H$ scalars

Table A1: **Shell Game backbone configurations.** SWA_k attends to the current and k previous chunks, the “-neg” suffix widens the transition spectrum from $[0, 1]$ to $[-1, 1]$, and GDP3 denotes Gated Delta Product with three Householder factors. L denotes depth and Params denotes the parameter count in millions.

Backbone	L	Mixer configuration	Transition range	Params (M)
Bidirectional DiT	12	None with full-clip attention	–	201.503
Causal DiT (training context, SWA_7)	12	None with SWA_7 attention	–	201.503
Causal DiT (SWA_1)	12	None with SWA_1 attention	–	201.503
RNN + SWA_1	8	Vanilla RNN with width 1152	nonlinear	162.988
GRU + SWA_1	8	GRU with width 1152	nonlinear	205.492
LSTM + SWA_1	8	LSTM with width 1152	nonlinear	226.744
Mamba2 + SWA_1	8	$d_{\text{state}} = 256, d_{\text{conv}} = 4$, expand 2, head dim. 72	positive	210.582
GDN + SWA_1	8	12 heads of width 96	$[0, 1]$	195.153
GDN-neg + SWA_1	8	12 heads of width 96	$[-1, 1]$	195.153
GDN2 + SWA_1	8	12 heads of width 96	$[0, 1]$	209.106
GDN2-neg + SWA_1	8	12 heads of width 96	$[-1, 1]$	209.106
GDP3 + SWA_1	8	12 heads of width 96 with 3 Householder factors	$[0, 1]$	237.989
GDP3-neg + SWA_1	8	12 heads of width 96 with 3 Householder factors	$[-1, 1]$	237.989
KDA + SWA_1	8	12 heads of width 96	$[0, 1]$	187.983
KDA-neg + SWA_1	8	12 heads of width 96	$[-1, 1]$	187.983

per layer. All LaCT variants use a dot-product binding loss (except for the MSE ablations), a base inner-loop learning rate of 10^{-3} , row-wise weight normalization, and one fast-weight update per clean chunk. The scratchpad carries M vectors of width D in each layer and updates them additively once per chunk.

Table A2: **LaCT and scratchpad state configurations.** L denotes depth and Params reports the total model parameter count in millions.

Variant	L	Heads / tokens	State structure per layer	State Size / layer	Params (M)
LaCT1	8	$H = 1$	$d = 1152$, three SwiGLU matrices	7,962,624	201.507
LaCT2	10	$H = 2$	$d = 576$, three SwiGLU matrices	3,981,312	209.559
LaCT4	11	$H = 4$	$d = 288$, three SwiGLU matrices	1,990,656	207.674
LaCT8	12	$H = 8$	$d = 144$, three SwiGLU matrices	995,328	213.850
LaCT16	12	$H = 16$	$d = 72$, three SwiGLU matrices	497,664	208.209
LaCT48	12	$H = 48$	$d = 24$, three SwiGLU matrices	165,888	205.555
Scratchpad1	12	$M = 1$	1×1152 additive memory	1,152	201.517
Scratchpad4	12	$M = 4$	4×1152 additive memory	4,608	201.558
Scratchpad64	12	$M = 64$	64×1152 additive memory	73,728	202.388
Scratchpad256	12	$M = 256$	256×1152 additive memory	294,912	205.042
Scratchpad1024	12	$M = 1024$	1024×1152 additive memory	1,179,648	215.659

D.2 TRAINING PROTOCOL

All models are trained from scratch with diffusion forcing, a diffusion variant of teacher forcing, using the Wan VAE as a tokenizer. Each five-latent-frame chunk is assigned an independently sampled noise level under a flow-matching schedule with shift $s = 3$. Causal masking is implemented with PyTorch’s FlexAttention backend, and the effective batch size is 16. Following long-context practice (Po et al., 2025), the attention, SSM, linear-attention, and RNN families use context mixing with probability 0.5, replacing a randomly sampled prefix with its clean latents and training on the remaining noisy chunks. LaCT and the scratchpad instead interleave noisy and clean copies of each chunk, updating the carried state on the clean copy of chunk t before denoising chunk $t + 1$, as in prior autoregressive video applications (Zhang et al., 2025a; Dalal et al., 2025). During inference, all models receive the two clean initial chunks containing the initial reveal and lowering, then generate the continuation autoregressively.

We optimize all models for 200K steps using NorMuon (Li et al., 2025) with a learning rate of 2×10^{-4} . Training uses mixed precision through `torch.autocast`, while the RNN, LSTM, and GRU layers remain in full precision for numerical stability. NorMuon consistently converges faster than AdamW in our experiments, as shown for Mamba2 + SWA_1 and the original Memory Maze in Fig. A1. Since the tracked state becomes visible only in the final reveal, we also tried upweighting the final-chunk loss by $10\times$ under AdamW, but it showed little effect.

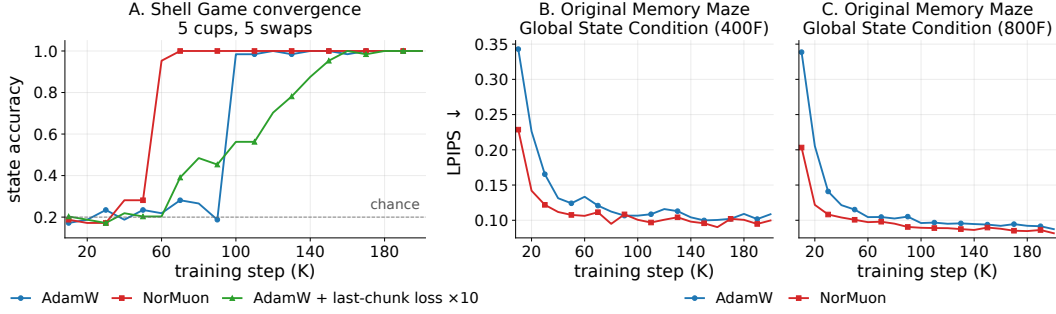


Figure A1: **NorMuon accelerates convergence.** All panels use Mamba2 + SWA1. (A) Shell Game state accuracy with NorMuon, AdamW, and AdamW with a $10\times$ final-chunk loss. (B, C) LPIPS on the original Memory Maze with global-state conditioning at 400 and 800 frames.

D.3 EVALUATION PROTOCOL

State accuracy is computed using a color-based ball detector on the final reveal frame. We divide the final reveal frame into five equal-width cup regions and predict the region with the strongest red-ball response among pixels satisfying $r - \max(g, b) > 0.18$. If no ball is detected, the episode is counted as incorrect, so an accuracy of 0.0 may indicate a failure to render the ball rather than selection of the wrong cup. We average accuracy over 64 episodes and report $\text{acc}@N$ for sequence lengths from the training horizon $N = 5$ to $N = 30$, against chance accuracy $1/5 = 0.2$, using the 200K-step EMA weights. SSIM, PSNR, and LPIPS are computed over the generated continuation against the ground-truth video using the same EMA checkpoints.

E MEMORY MAZE EXPERIMENT DETAILS

Setup. Following Po et al. (2025), all Memory Maze models train at 128×128 resolution on 400-frame clips, using the same backbone families as the Shell Game. Each dataset contains 29K training and 1K validation trajectories of 1001 frames. We precompute video latents with the frozen Wan2.1 VAE using $4\times$ temporal and $8\times$ spatial compression. All backbones are trained from scratch with an effective batch size of 16 and a learning rate of 2×10^{-4} , and we report the EMA weights at step 200K. Models on the original benchmark use AdamW, while models on the two deterministic datasets use NorMuon (Li et al., 2025). For global-state conditioning, a two-layer MLP projects the normalized 2D agent position and 2D heading direction to D . For local-action conditioning, learned embeddings represent the six discrete controls `no-op`, `forward`, `turn-left`, `turn-right`, `forward-left`, and `forward-right`, with the initial pose provided only at the first latent step. In both cases, four frame-wise conditions are packed into each latent-frame embedding, added to the diffusion timestep embedding, and injected into every block through AdaLN.

Deterministic dataset. The original benchmark drives the agent through a MuJoCo simulation, so the same discrete action produces a variable displacement depending on its physical state. Our deterministic rebuild replays the original 9×9 layouts with grid kinematics, where a forward move advances exactly one cell and a turn rotates exactly 90 degrees. We used a waypoint navigation-based policy to make agents move through target points (in shortest paths), and tried our best to match the coverage statistics of the public dataset. Both the original and our rebuilt dataset contain 29K training and 1K validation episodes of 1001 frames each. In the no-collision variant the policy never attempts a blocked move, so the pose is a pure function of the action stream. In the collision variant, the policy may attempt to move into walls, and a blocked move leaves the pose unchanged, making the task harder because the model must infer from visual features whether each forward action succeeded and update the pose accordingly.

Evaluation. We evaluate a 220-frame continuation under two context lengths. The first uses 177 clean exploration frames, yielding a 397-frame sequence that matches the training horizon. For comparison with Po et al. (2025), we also use 577 clean frames, closely following their evaluation

Table A3: **Original Memory Maze with AdamW after 200K training steps.** Global State uses the ground-truth agent pose, while Local Action uses the discrete action sequence. Each metric scores the generated 220-frame continuation from 177 and 577 clean context frames. Due to RoPE extrapolation issues, Causal DiT uses its training-context SWA19 window, spanning 20 chunks or 397 frames.

Backbone	Global State						Local Action					
	400 frames			800 frames			400 frames			800 frames		
	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓
Bidirectional DiT	0.928	27.79	0.118	0.912	26.32	0.142	0.842	21.96	0.210	0.733	17.13	0.365
Causal DiT (training context, SWA19)	0.933	30.39	0.102	0.938	29.25	0.094	0.826	21.85	0.227	0.846	22.05	0.194
GRU + SWA1	0.914	27.23	0.118	0.918	27.76	0.113	0.780	19.02	0.299	0.824	21.20	0.232
Mamba2 + SWA1	0.932	29.35	0.109	0.943	32.18	0.087	0.818	21.20	0.239	0.849	23.43	0.192
GDN + SWA1	0.931	29.24	0.102	0.935	29.42	0.097	0.828	20.90	0.223	0.849	22.13	0.193
GDN-neg + SWA1	0.926	27.87	0.111	0.933	29.40	0.097	0.827	20.86	0.225	0.854	22.81	0.185
LaCT8 + SWA1	0.934	30.47	0.095	0.937	31.46	0.086	0.827	22.21	0.230	0.864	24.29	0.174

Table A4: **Deterministic Memory Maze with NorMuon after 200K training steps.** The conditioning and metric conventions follow Tab. A3.

Backbone	Global State						Local Action					
	400 frames			800 frames			400 frames			800 frames		
	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓	SSIM↑	PSNR↑	LPIPS↓
<i>No collision</i>												
Causal DiT (training context, SWA19)	0.945	30.51	0.075	0.948	32.09	0.072	0.934	29.02	0.087	0.935	30.42	0.083
GRU + SWA1	0.940	31.48	0.082	0.944	32.56	0.077	0.883	22.73	0.177	0.899	24.15	0.145
Mamba2 + SWA1	0.942	31.43	0.083	0.949	33.53	0.069	0.928	29.75	0.097	0.936	31.56	0.082
GDN + SWA1	0.942	30.20	0.080	0.948	33.22	0.071	0.929	30.37	0.094	0.936	31.96	0.082
GDN-neg + SWA1	0.942	30.13	0.080	0.948	33.25	0.071	0.932	30.83	0.089	0.938	32.16	0.080
LaCT8 + SWA1	0.946	32.25	0.077	0.952	33.94	0.065	0.943	31.47	0.081	0.952	33.57	0.066
<i>With collision</i>												
Causal DiT (training context, SWA19)	0.911	27.84	0.112	0.923	28.69	0.091	0.859	24.62	0.173	0.884	24.84	0.137
GRU + SWA1	0.892	25.65	0.140	0.911	28.73	0.108	0.765	17.57	0.331	0.792	18.40	0.285
Mamba2 + SWA1	0.910	27.63	0.113	0.926	30.84	0.085	0.827	21.14	0.230	0.863	24.54	0.171
GDN + SWA1	0.907	24.82	0.121	0.929	30.90	0.083	0.853	23.76	0.187	0.878	26.39	0.149
GDN-neg + SWA1	0.908	25.11	0.119	0.929	30.58	0.085	0.863	24.98	0.171	0.895	28.24	0.122
LaCT8 + SWA1	0.917	28.42	0.103	0.937	32.05	0.075	0.878	25.16	0.158	0.906	28.24	0.116

with 576 context frames and a 224-frame rollout. Despite training only at 400F, most recurrent variants, including LaCT, improve with the longer prefix, suggesting that their learned state updates remain effective over longer observed trajectories. We refer to the two settings as 400F and 800F for simplicity. At both lengths, the causal DiT uses its training-context SWA19 window, spanning 20 chunks or $4(5 \times 20) - 3 = 397$ decoded frames. We report SSIM, PSNR, and LPIPS using the 200K-step EMA weights. Qualitative rollouts for both conditioning schemes are shown in Appendix H. On the original Memory Maze at 800F, our causal DiT reaches a PSNR of 29.25, compared with 28.8 for the full-context causal DiT of Po et al. (2025), while our Mamba2 + SWA1 reaches 32.18, compared with 28.2 for their SSM model. Although their models receive additional 800F training, ours reach higher PSNR after training only at 400F, likely reflecting advances in the DiT scaffold, latent space, and optimization.

Results. Tabs. A3 and A4 report results under global-state and local-action conditioning. Most recurrent backbones benefit from the longer observed context despite being trained only at 400F.

Prior long-context Memory Maze studies primarily use global-state conditioning, which provides the exact agent pose at every frame (Po et al., 2025; Chen et al., 2025). Each observation is therefore effectively tagged by its pose, allowing the recurrent state or KV cache to act as a pose-indexed visual memory. Mamba2 and LaCT perform strongly in this setting, showing that both mechanisms can store and retrieve visual information over long trajectories. Local-action conditioning instead provides the initial pose followed by discrete controls. The model must track its pose before retrieving the visual memory associated with the current location. The task therefore combines state tracking with visual retrieval rather than retrieval alone. On the original Memory Maze, we found that all

models perform substantially worse under local-action conditioning. Further inspection suggested that the models accumulate localization error over long trajectories because the MuJoCo dynamics can produce different displacements for the same discrete action. Although wall collisions are rare in the released trajectories, these MuJoCo dynamics make accurate dead reckoning from the action sequence difficult.

We designed the deterministic variants to isolate whether this drop comes from local-action conditioning itself or from drift in the MuJoCo dynamics. Without collisions, each action applies a fixed translation or rotation, making the pose a deterministic function of the initial pose and action sequence. Pose tracking can then be expressed as a prefix-sum-style composition of fixed action-induced transformations. The global-state and local-action results become much closer in this setting, indicating that local-action conditioning itself is not the main source of the original gap. At 800F, LaCT reaches the same SSIM of 0.952 under both global-state and local-action conditioning, essentially closing the gap between the two.

Allowing collisions makes the transition also depend on what the model sees. A blocked forward move leaves the pose unchanged, so the action sequence alone no longer determines the trajectory, and the model has to judge from the frames whether each move succeeded before updating its state. This is no longer the action-only prefix accumulation of the collision-free variant and all backbones’ performance drops relative to their no-collision variant, with the drop being larger under local-action conditioning. Global-state conditioning suffers too because wall contacts make the visual transitions harder, but local-action conditioning on top of that has to resolve their effect on pose.

LaCT is the strongest backbone across most 800F settings, particularly under local-action conditioning. We hypothesize that LaCT benefits from its expressive nonlinear fast-weight update, which may better represent transitions that combine actions with visual evidence. In contrast, widening the linear transition spectrum to include negative eigenvalues yields no consistent gain, with GDN-neg performing similarly to GDN. This suggests that Memory Maze places different structural demands on the state transition and does not show the clear benefit from negative eigenvalues observed in the Shell Game.

F 3D BLOCK WORLD EXPERIMENT DETAILS

Setup. We use the textured 3D Block World of [Lillemark et al. \(2026\)](#), a MiniWorld-based room whose blocks keep moving on their own, so parts of the scene continue to evolve while the agent’s camera is turned away. The agent moves under the four discrete controls `no-op`, `forward`, `turn-left`, and `turn-right`, with each turn rotating the view by exactly 90 degrees. As in the Memory Maze local-action protocol, the normalized initial pose is injected only at the first latent step. Episodes contain 500 frames at 128×128 resolution. We found that the Wan VAE’s $4\times$ temporal compression handles abrupt 90-degree view changes poorly, so we use FloWM’s framewise MAE-ViT-based image VAE instead. It produces 8-channel latents with $16\times$ spatial compression and no temporal compression, aligning each action with one latent frame. Each autoregressive chunk spans five latent frames, and all backbones train from scratch on 160-frame clips with NorMuon at learning rate 2×10^{-4} and an effective batch size of 32 for 200K steps. As on the Shell Game, the models train with a clean prefix covering a random half to all of the leading chunks, while LaCT interleaves the clean and noisy copies of each chunk and uses no such prefix (Appendix D.2).

Evaluation. Evaluation feeds a clean 80-frame prefix and scores the generated continuation with SSIM, PSNR, and LPIPS using the 200K-step EMA weights. The 160-frame setting stays at the training length and generates 80 frames, while the 320-frame setting generates 240 frames and probes rollouts well beyond it.

Results. At roughly 200M parameters, all backbones model the training-length rollouts reasonably well, and most remain visually coherent at 320 frames. Among the models restricted to SWA1, LaCT performs substantially better than the other recurrent backbones. At 320 frames, LaCT reaches a PSNR of 29.36 and an LPIPS of 0.0288, compared with 21.32 and 0.1060 for Mamba2. LaCT has no explicit state target or architectural constraint for tracking out-of-view objects, but its nonlinear fast-weight update may provide a more expressive carrier for such information. By comparison,

GDN performs similarly to Mamba2, and widening its transition spectrum with GDN-neg yields little additional benefit.

In our experiments, increasing the local attention window has a larger effect than changing the recurrent update. For Mamba2, expanding attention from SWA1 to SWA10 improves the 320-frame PSNR from 21.32 to 32.29 and reduces LPIPS from 0.1060 to 0.0167 (Tab. A6). We find that longer recent context is particularly helpful here because framewise encoding and abrupt 90-degree turns make adjacent observations less redundant. The improvement peaks at SWA10, with performance declining again at SWA15 and SWA20.

We also note that the windowed baselines are strong relative to the results reported by Lillemark et al. (2026). Their textured Block World evaluation uses 70 context frames followed by 210 generated frames, where FloWM, DFoT, and DFoT-SSM reach PSNR scores of 30.33, 20.43, and 19.15. In our slightly longer setting with 80 context frames and a 240-frame continuation, the roughly 200M-parameter Causal SWA5, Mamba2 with SWA10, and LaCT with SWA1 reach 37.60, 32.29, and 29.36. These results suggest that much of the task can be addressed through backbone scaling and sufficient recent context. The open-room layout may contribute to this behavior because moving objects often remain visible or return within the attention window.

Table A5: **Textured 3D Block World backbone comparison under local-action conditioning after 200K training steps.** All metrics use the EMA weights. SWA k attends to the current and k previous chunks, with five framewise latents per chunk.

Backbone	k	160 frames			320 frames		
		SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$
Causal DiT (training context, SWA31)	31	0.9944	42.03	0.0058	0.9630	24.98	0.0495
Causal DiT (SWA5)	5	0.9922	40.02	0.0088	0.9907	37.60	0.0111
Causal DiT (SWA1)	1	0.9376	22.02	0.0877	0.9104	20.21	0.1284
Mamba2 + SWA1	1	0.9473	23.88	0.0745	0.9255	21.32	0.1060
GDN + SWA1	1	0.9469	23.96	0.0747	0.9250	21.52	0.1067
GDN-neg + SWA1	1	0.9478	24.42	0.0723	0.9260	21.36	0.1037
LaCT8 + SWA1	1	0.9908	38.26	0.0114	0.9791	29.36	0.0288

Table A6: **Mamba2 SWA sweep on Textured 3D Block World under local-action conditioning after 200K training steps.** The metric and SWA conventions follow Tab. A5.

k	160 frames			320 frames		
	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$
1	0.9473	23.88	0.0745	0.9255	21.32	0.1060
2	0.9738	29.58	0.0358	0.9476	23.62	0.0744
3	0.9885	35.98	0.0150	0.9762	28.87	0.0338
5	0.9921	39.40	0.0093	0.9837	30.29	0.0234
10	0.9926	40.21	0.0083	0.9870	32.29	0.0167
15	0.9940	40.67	0.0066	0.9746	27.07	0.0332
20	0.9939	40.57	0.0067	0.9503	23.27	0.0645

G ADDITIONAL DISCUSSION AND LIMITATIONS

Global State Conditioning vs Local Action Conditioning. Many recent video world models condition generation directly on global camera poses or prescribed trajectories (Ren et al., 2025b; Huang et al., 2025a; Wu et al., 2025a; Nam et al., 2026; Yi et al., 2026), with interactive use mapping keyboard and mouse inputs into camera motion through a calibrated controller (Nam et al., 2026). This supports precise viewpoint control and pose-indexed retrieval, but it assumes the camera state is available and factors self-localization out of the learning problem. Raw discrete actions are cheaper to obtain, and a model conditioned on them must instead infer how local controls change the global state. Action-conditioned models are common in interactive gaming domains (He et al., 2025; Wang et al., 2026; Savva et al., 2026), yet we still think pose recovery from an initial pose and a stream

of local actions has received little direct study. Our original Memory Maze runs already show this. Under MuJoCo the same action can move the agent by different amounts, so integrated poses drift, and no local-action-conditioned backbone recovers global-state fidelity. We believe recovering the physics engine’s state from observations and discrete local actions alone, and correcting its drift over long horizons, remains an open problem.

State Carriers and Learnability. An LLM can append a token that snapshots the current state and read it back at the next step (Merrill & Sabharwal, 2024), and supervised traces or sequence-level rewards can teach that protocol. Pure video training provides no such target. In our setup, flow matching trains each chunk toward a reconstruction latent of the frozen Wan VAE, which carries no label for the hidden state, so generated frames cannot serve as a scratchpad (Appendix A). Large systems are beginning to split the problem accordingly, with Cosmos 3 routing reasoning through a dedicated autoregressive tower while a separate diffusion generator renders from its representations (Aditi et al., 2026). Keeping the state inside the video backbone is what our recurrent variants test, but we often found expressivity alone is not enough. The clearest case is the nonlinear RNNs, which solve the toy S_5 task in a single layer, yet inside the video stack the vanilla RNN never fits even the training length and the GRU fits it but still falls toward chance beyond it. LaCT (SwiGLU) with a single fast-weight head fails to extrapolate even though it carries the largest state, while more heads track better with less state, pointing to factorized updates or easier optimization rather than raw capacity. The benefit has limits of its own. For example, raising the LaCT head count from 8 to 48 lowered visual fidelity on the deterministic Memory Maze, suggesting that maintaining video quality needs a sufficient head dimension, so we keep LaCT8 as the reference throughout. The inner architecture, update loss, normalization, and update schedule all shape whether a formally expressive fast-weight model learns a useful transition, so a failing run may reflect a learning difficulty rather than a representational limit.

Effect of Chunk Size in LaCT. LaCT performs one serial fast-weight update per chunk, so a sequence of length L processed in chunks of size c receives roughly L/c serial updates, and the swaps inside a chunk must be composed by the fixed-depth chunk processor before the state is written (Sec. B). As the toy depth sweep in Fig. 1 shows, a Transformer needs more layers to compose a longer swap chain, so larger chunks push work from the recurrence back onto backbone depth. We tried a chunk-size ablation for TTT layers in the toy setting, but small chunks unroll the fast-weight update into a very long gradient chain, and training became too slow and unstable to finish the sweep. The runs we could finish still pointed the expected way, with smaller chunks solving the task at lower depth. On video the same cost stretches across hundreds of latent frames, so we did not attempt a controlled sweep there. Matrix-state linear attention is easier to move along this axis, since its outer-product writes can be defined at token, frame, or group granularity while keeping optimized parallel kernels, though we did not test these granularities here. Nonlinear fast weights have no such kernels, and in our runs LaCT and the scratchpad train markedly slower than the attention and linear-attention baselines, both because interleaving clean and noisy copies of each chunk doubles the effective sequence length and because the carried state serializes training across chunks. Nonetheless, they are still the easier tool for research, since the apply and update operations are written directly in PyTorch and can be rearranged or replaced.

Limitations. The Shell Game maps directly onto S_5 state tracking, but as Sec. 6 shows, more general settings have no such clean correspondence. There, part of the state must be corrected from what the model observes (often not annotated by action inputs), and we do not yet know which kind of state tracking is the right abstraction for video world models. The mechanisms that work here are validated only through length extrapolation on synthetic tasks, at one model scale around 200M parameters, with one flow-matching recipe. Whether any of this transfers to real-world video or survives at the scale of modern video world models is untested. Nonetheless, video models remain a scalable way to learn world dynamics, and we believe understanding and learning the structure of the diverse state updates and implicit states they require is a promising research direction.

H QUALITATIVE SAMPLES

We show generated rollouts for the three experiments in the paper. Each figure follows one episode, and all rows share the same context frames before generating the rest independently. Fig. A2 shows the Shell Game and Fig. A3 illustrates the textured 3D Block World experiment. For the Memory

Maze, Figs. A4 to A9 show the original and deterministic datasets (without collision and with collision) under the 800F evaluation of Appendix E, with both conditioning schemes on the same episode of each dataset.



Figure A2: **Shell Game rollout at twice the training length.** Models trained at $N=5$ are rolled out over $N=10$ swaps. Rows are the ground truth and four backbones, where the causal DiT keeps its training-context SWA7⁵ window and the recurrent backbones use SWA1. At the final reveal, GDN-neg and LaCT8 land on the same cup as the ground truth, while the causal DiT and GDN do not.

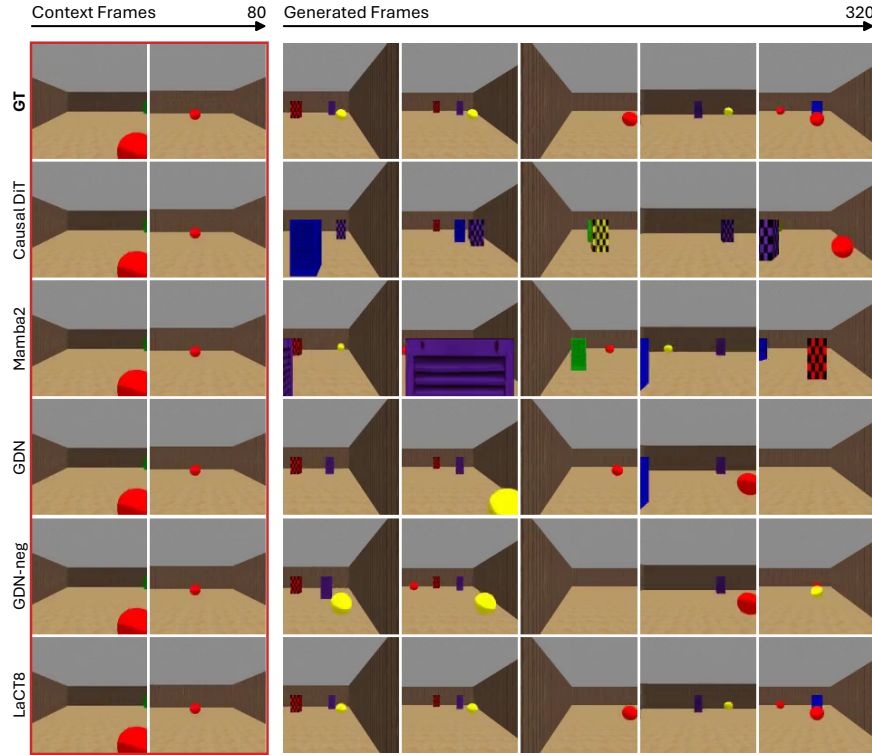


Figure A3: **Textured 3D Block World under local-action conditioning.** The rollout uses an 80-frame clean prefix and a 240-frame continuation (Tab. A5). All rows use SWA1, so they differ only in the recurrent update. LaCT8 keeps the blocks and the ball near their positions in the ground truth, while the other backbones render a coherent room whose contents have drifted.

⁵Every causal window misses the cup, but frames get rougher as the window widens. A training episode is eight chunks and starts with two reveal chunks, so only SWA7 keeps them in view for all of training and never learns to run without them, which is why it scores lowest in Fig. 4 and blurs the later frames here beyond $N=5$.

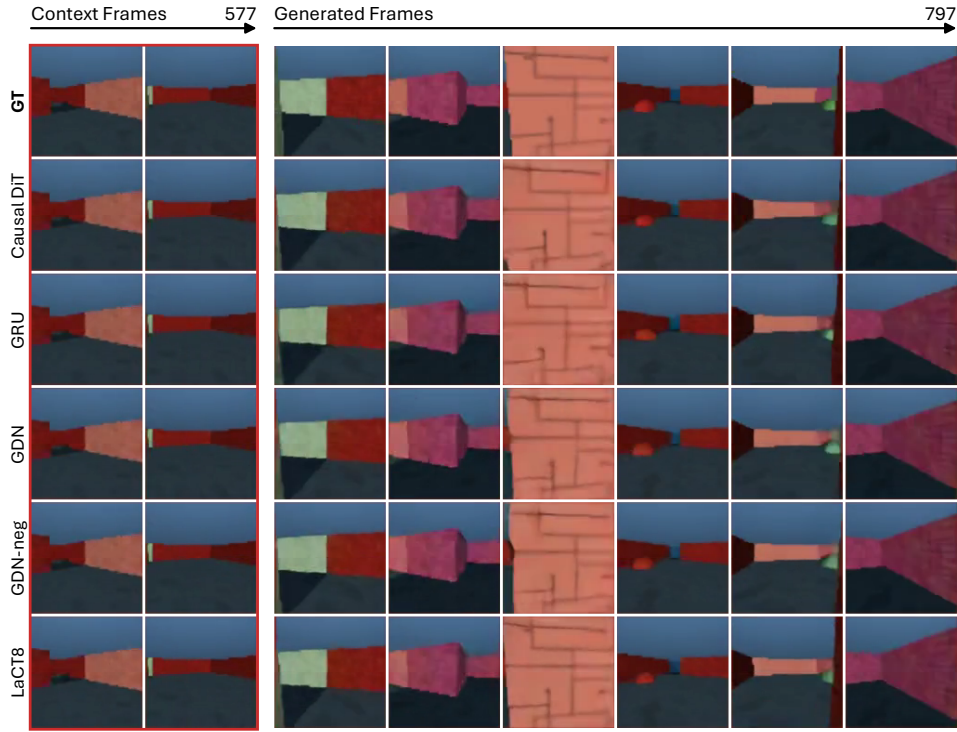


Figure A4: **Memory Maze under global-state conditioning.** Rows are the ground truth and the backbones of Tab. A3 (original Memory Maze), where the causal DiT uses its training-context SWA19 window and the recurrent backbones use SWA1.

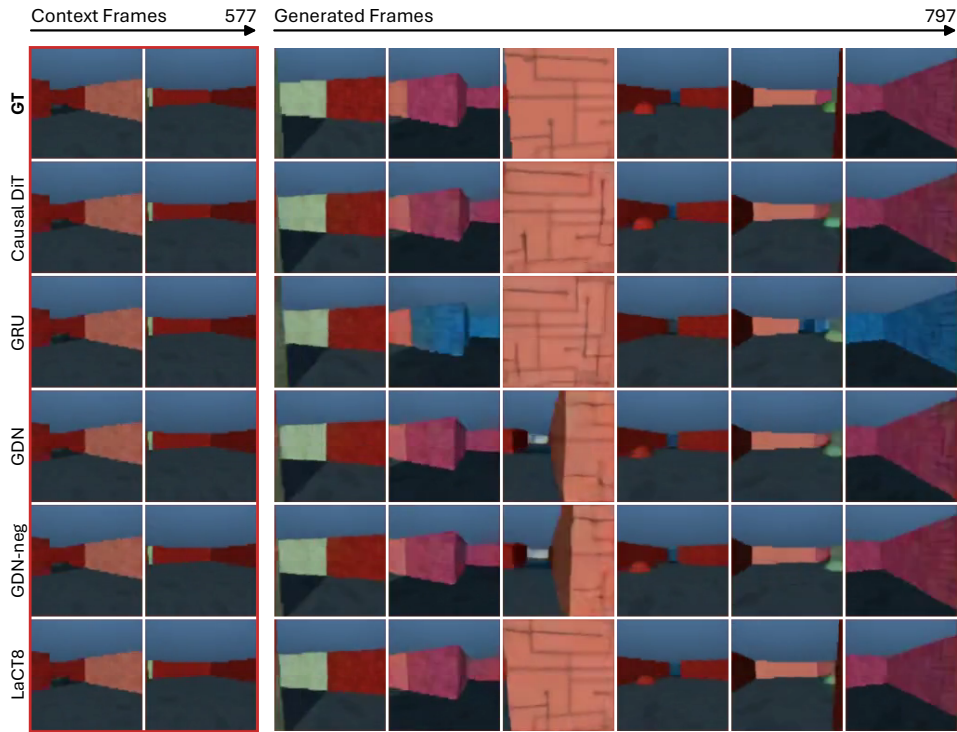


Figure A5: **Memory Maze under local-action conditioning.** The episode and frames are those of Fig. A4.

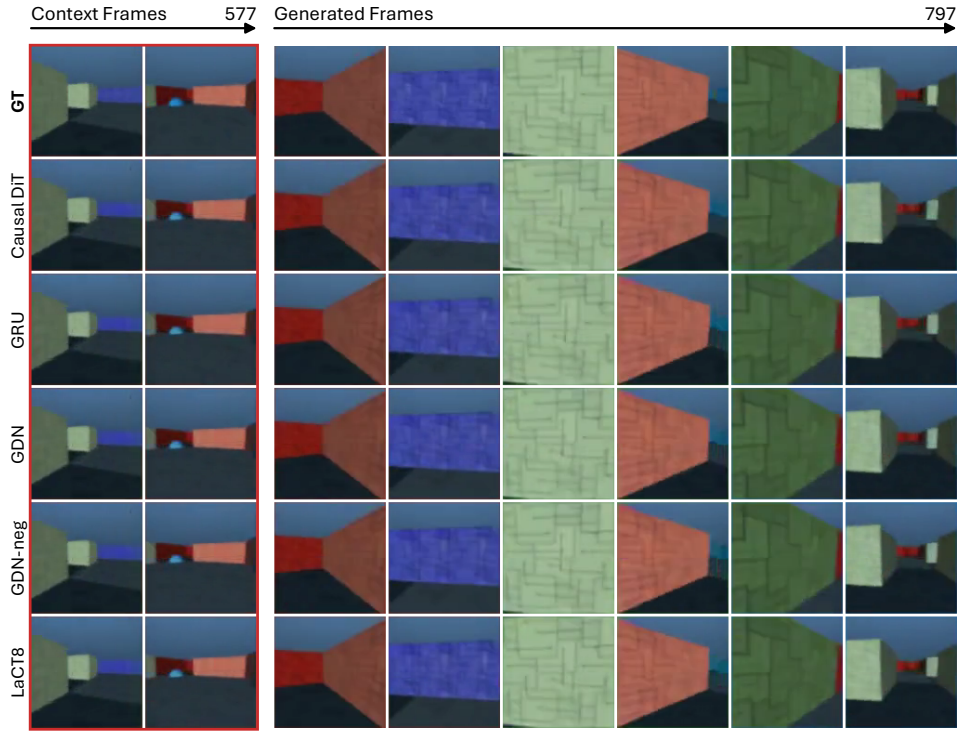


Figure A6: **Deterministic Memory Maze without collisions, global state.** Conventions follow Fig. A4, with the backbones of Tab. A4 (deterministic Memory Maze without collisions).

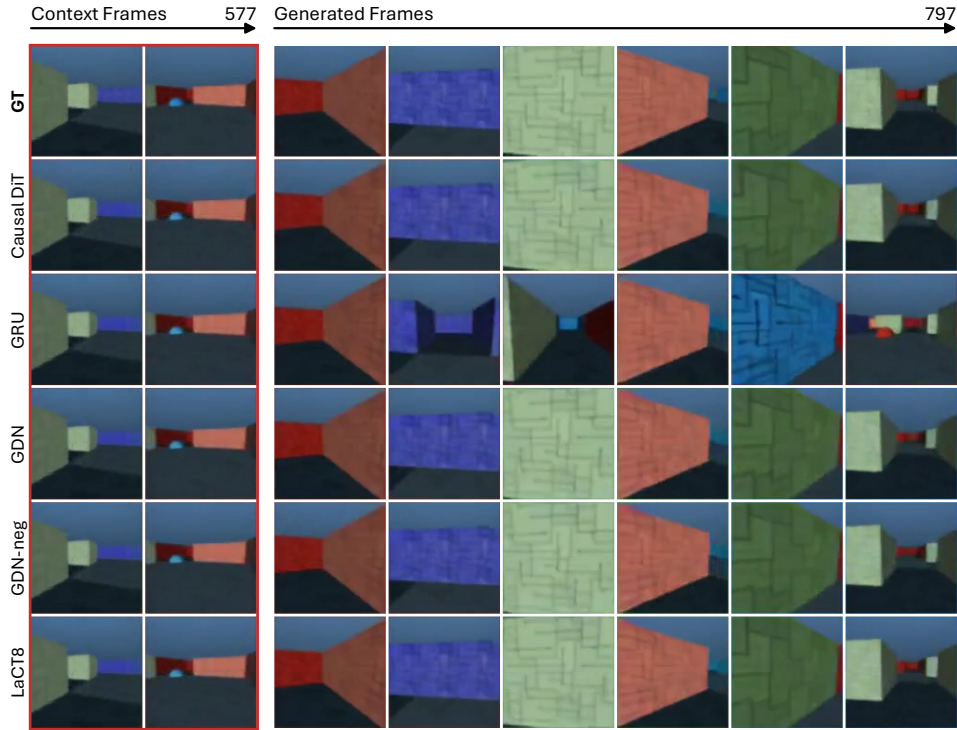


Figure A7: **Deterministic Memory Maze without collisions, local action.** Pose is a pure function of the action stream in this variant, and every backbone except the GRU stays close to the ground truth through the rollout.

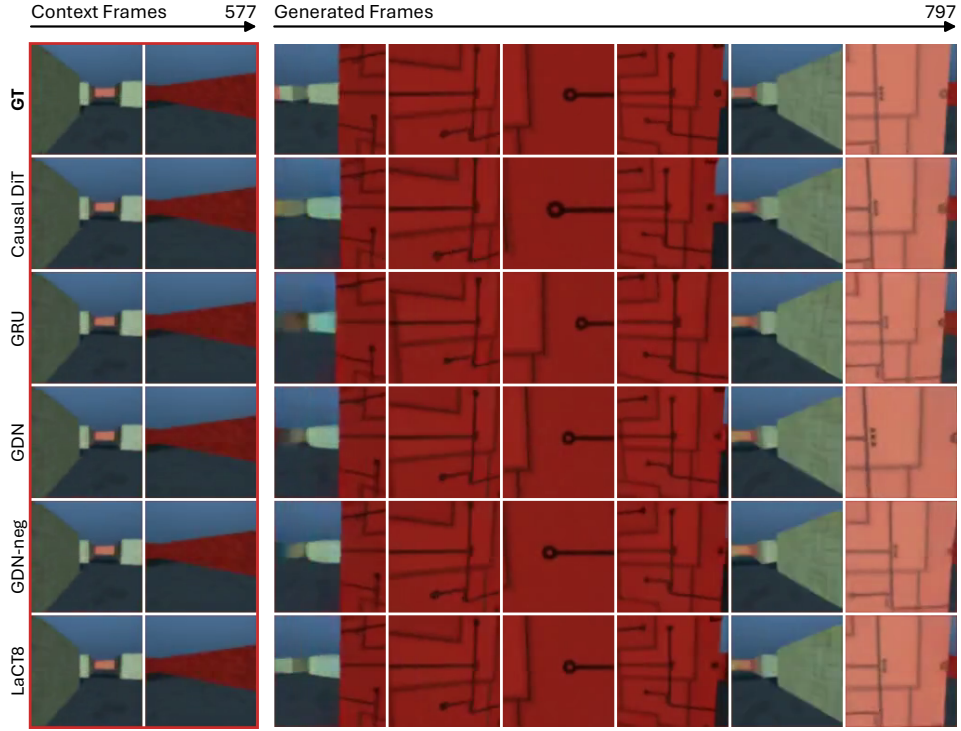


Figure A8: **Deterministic Memory Maze with collisions, global state.** Conventions follow Fig. A4, with the backbones of Tab. A4 (deterministic Memory Maze with collisions).

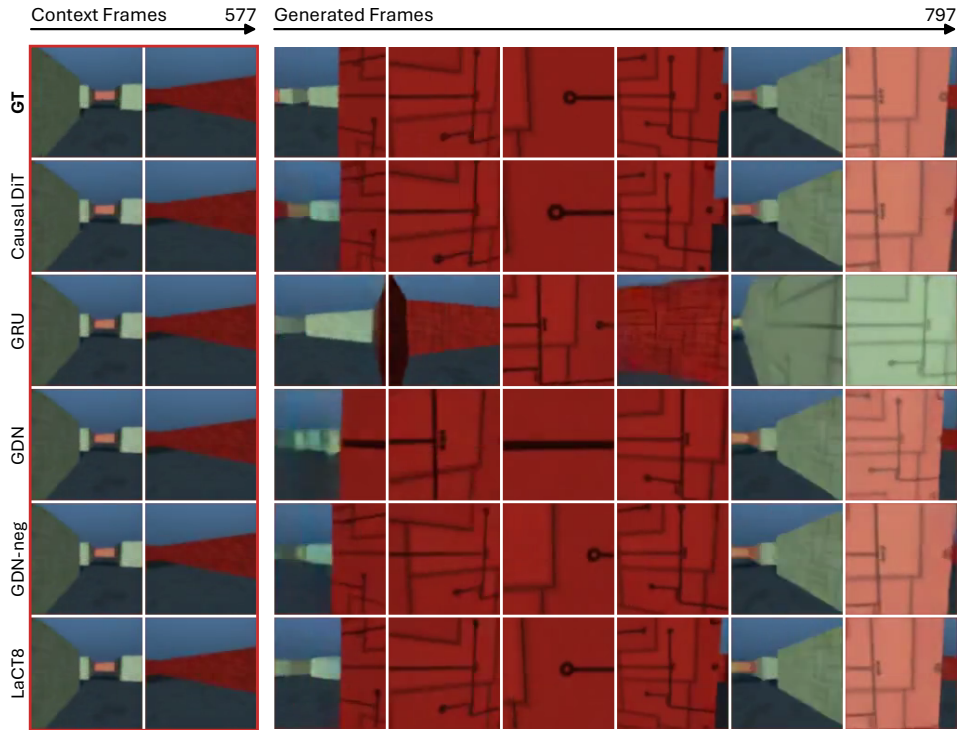


Figure A9: **Deterministic Memory Maze with collisions, local action.** A blocked move leaves the pose unchanged, so the update depends on the layout the model cannot see.